

YAMBA — CAHIER DE RECETTE . TÂCHES PLANIFIÉES, RELAIS D'ÉVÉNEMENTS ET CONSOMMATEURS

Yamba — documentation générée le 06/09/2026 depuis `docs/recette/RECETTE-04-CRONS.md`

Table des matières

1. Objet et périmètre
 - 1.1 Pourquoi ce cahier existe
 - 1.2 Ce que ce cahier couvre
 - 1.3 Ce que ce cahier ne couvre pas
 - 1.4 Les documents de référence
 - 1.5 Divergences documentaires déjà connues
2. Prérequis
 - 2.1 L'infrastructure locale
 - 2.2 Les sujets du courtier
 - 2.3 La boîte aux lettres de recette
 - 2.4 Le jeu d'essai
 - 2.5 Lire les battements
 - 2.6 Couper une tâche
 - 2.7 Forcer un passage : les trois méthodes
 - 2.8 Lire l'état de l'outbox
 - 2.9 Lire et consommer le courtier
3. Conventions
 - 3.1 Numérotation
 - 3.2 Verdicts
 - 3.3 Gravité des anomalies
 - 3.4 La fiche type d'une tâche
 - 3.5 Les treize tâches, vue d'ensemble
4. Les treize tâches, une par une
 - 4.1 Complétion des trajets — complete-trips
 - 4.2 Expiration des demandes à 24 h — expire-bookings
 - 4.3 Versement à J+4, rejeu et rappel — payout-bookings
 - 4.4 Alertes de seuil — ops-alerts
 - 4.5 Notation : relances et révélation — rating
 - 4.6 Récapitulatif quotidien — ops-digest
 - 4.7 Relance des messages non lus — unread-reminder
 - 4.8 Purge des conversations — conversation-retention
 - 4.9 Purge des événements publiés — outbox-retention (deal et message)
 - 4.10 Effacement du destinataire — recipient-redaction
 - 4.11 Conservation générale — retention (notification-service)
 - 4.12 Rappels d'inscription Voyageur — onboarding-reminder
5. Relais d'événements
 - 5.1 Ce qu'il faut savoir avant de tester
 - 5.2 Outils de ce chapitre
6. Consommateurs
 - 6.1 Outils de ce chapitre
7. Battements et moniteur externe
 - 7.1 La mécanique
 - 7.2 La carte des battements sortants
8. Sécurité et conformité
9. Consignation
 - 9.1 Tableau de suivi
 - 9.2 Fiche d'anomalie
 - 9.3 Critères de sortie
 - 9.4 Nettoyage après campagne

Périmètre testé : les **treize tâches planifiées** des cinq services, les **deux relais d'outbox** (`deal-service`, `message-service`), les **deux consommateurs** du `notification-service`, et les **battements** qui prouvent que tout cela tourne encore. Version du code de référence : branche `feat/f3-messaging-admin` (`base dev`), état au 06/09/2026. Document **exécutable par un testeur seul**. Chaque scénario se conclut par un verdict binaire : **conforme** ou **non conforme**.

1. Objet et périmètre

1.1 Pourquoi ce cahier existe

Les trois autres cahiers de recette regardent l'application par l'écran. Un testeur clique, l'application répond, et l'écart se voit tout de suite. Ce cahier-ci regarde la partie du système que **personne ne voit fonctionner**.

Une tâche planifiée qui ne tourne plus ne produit **aucun symptôme immédiat** :

- elle ne lève pas d'erreur — elle ne s'exécute simplement pas ;
- elle n'apparaît dans aucun journal — un cron qui ne démarre pas n'écrit pas de ligne ;
- elle n'a pas d'écran — aucun utilisateur ne va constater qu'un versement n'est pas parti ;
- et son effet est **différé** : le versement à J+4 manquant se découvre au mieux le cinquième jour, par une réclamation.

Le projet en a déjà fait les frais. Le cron de rappel d'inscription Voyageur (`onboarding-reminder.cron.ts`) existait depuis l'origine du dépôt, complet, testé, commenté — et n'était **appelé nulle part**. Aucun rappel n'est jamais parti pendant des mois, sans qu'aucun test, aucun journal et aucun écran ne le signale. Le correctif (A148) l'a branché dans `apps/auth-service/src/main.ts`. C'est exactement le type de panne que ce cahier doit rendre visible.

Le même raisonnement vaut pour le relais d'événements et pour les consommateurs. Le relais est le seul chemin entre une transition métier (écrite dans Mongo) et une notification ou un email. S'il s'arrête, l'application continue de répondre 200 à tout le monde : les réservations s'acceptent, les remises se valident, et **rien ne part**. Les événements s'accumulent dans la collection `OutboxEvent`, invisibles, jusqu'à ce qu'un membre demande pourquoi il n'a jamais reçu son email.

Ce cahier donne donc, pour chaque tâche, deux choses que la documentation technique ne donne pas :

1. **Comment la déclencher à la demande**, sans attendre 03:40 du matin ni cinq minutes de tick ;
2. **Comment prouver qu'elle a fait son travail** — un effet mesurable en base, un email lisible, un message consommable sur le courtier, un battement à jour dans Redis.

1.2 Ce que ce cahier couvre

Domaine	Objets testés
Tâches planifiées	Les treize <code>apps/*/src/cron/*.ts</code> , leur horaire réel, leur interrupteur, leur garde de non-chevauchement
Règles métier des tâches	Les services appelés : expiration, versement, notation, alertes, purges, effacement du destinataire, relances
Relais d'événements	<code>apps/deal-service/src/relay/outbox-relay.ts</code> et <code>apps/message-service/src/relay/messaging-relay.ts</code> : bail, filtrage par type d'agrégat, parking, redémarrage du courtier
Consommateurs	<code>apps/notification-service/src/consumer/booking-events.consumer.ts</code> et <code>messaging-events.consumer.ts</code> : dédoublement, offsets, rattrapage

Domaine	Objets testés
Observabilité	Battements Redis <code>yamba:cron:*</code> , page « État des services » du back-office, battement sortant vers un moniteur externe
Conformité	Ce que les purges ne doivent jamais supprimer, ce qu'un payload ne doit jamais contenir, à qui un email ne doit jamais partir

1.3 Ce que ce cahier ne couvre pas

- Les parcours **membres** (publication d'un trajet, réservation, paiement, remise, notation, messagerie) : voir `docs/recette/RECETTE-01-MEMBRE.md`.
- Le **back-office** : les 23 écrans, la connexion en deux étapes, la matrice des permissions, le journal d'audit : voir `docs/recette/RECETTE-02-ADMIN.md`.
- Les **contrats d'API** pris isolément (codes de retour, schémas OpenAPI, en-têtes) : voir `docs/recette/RECETTE-03-API.md`.

Quand une tâche planifiée a un effet visible côté membre (un versement qui arrive, un avis qui se dévoile, une conversation qui disparaît), ce cahier exige la vérification de l'effet mais renvoie au cahier 01 pour le parcours d'écran.

1.4 Les documents de référence

Document	Ce qu'on y trouve
<code>docs/livrables/04-YAMBA-DOCUMENTATION-TECHNIQUE.md</code> , chapitre 9	Le tableau transverse des treize tâches, les variables d'environnement, le runbook du moniteur externe
<code>context/YAMBA-REGISTRE-DECISIONS-ROADMAP-v1.3.md</code>	Les décisions D2 (outbox), D49 (versement), D53 (notation), D59 (alertes), D61 (messagerie), D63 (effacement du tiers), D64 (conservation, battements), D70 (moniteur externe)
<code>packages/libs/api-contracts/src/admin/platform-settings.schema.ts</code>	Le catalogue des paramètres : durées de conservation, seuils d'alerte, délais de relance
<code>context/YAMBA-PARAMETRES.md</code>	Le même catalogue, en français, pour l'exploitant
<code>packages/libs/retention/index.ts</code>	Les règles pures de conservation (ce qui est purgeable, ce qui ne l'est jamais)
<code>packages/libs/redis/cron-heartbeat.ts</code>	La mécanique des battements et du battement sortant

Règle de précedence : le code fait foi. Un écart entre un document et le comportement observé se consigne comme anomalie **documentaire** (gravité mineure). Un écart entre le comportement observé et le code se consigne comme anomalie **fonctionnelle**.

1.5 Divergences documentaires déjà connues

Ces trois écarts ont été relevés à la rédaction de ce cahier, entre le chapitre 9 de la documentation technique et le code réel. Ils sont **déjà consignés** : inutile de les rouvrir en anomalie, mais il faut les connaître pour ne pas tester la mauvaise chose.

#	Le document dit	Le code dit	Conséquence pour le testeur
DIV-1	Le cron <code>onboarding-reminder</code> est « défini mais jamais démarré », sans interrupteur	<code>apps/auth-service/src/main.ts</code> (l. 64-72, A148) le démarre après le <code>listen</code> , coupable par <code>ONBOARDING_REMINDER_CRON_ENABLED=false</code>	Le chapitre 12 de ce cahier teste ce cron : il doit battre et envoyer

#	Le document dit	Le code dit	Conséquence pour le testeur
DIV-2	La ligne outbox-retention du tableau porte « trip / deal / message »	Seuls apps/deal-service et apps/message-service ont ce cron ; apps/trip-service/src/main.ts ne démarre que complete-trips	Ne pas chercher un battement trip-service:outbox-retention : il n'existe pas
DIV-3	L'en-tête de payout-bookings.cron.ts annonce un rejeu des versements « < 10 essais »	retryFailedPayouts n'a aucun plafond de tentatives depuis C-PR5 / D58 / A111 ; l'espacement se fait par payoutNextRetryAt	Un versement en échec est retenté indéfiniment, de plus en plus espacé (voir CRON-PAYOUT-5)

2. Prérequis

Rien de ce cahier n'est jouable sans cette section. Elle se fait **une fois**, au début de la campagne.

2.1 L'infrastructure locale

```
cd /Users/gomab/Documents/Dev/Projects/yamba-app
docker compose up -d
docker ps --format '{{.Names}}\t{{.Status}}'
```

Attendu : deux conteneurs, yamba-redpanda (courtier, ports 9092 et 9644) et yamba-mailpit (boîte aux lettres de recette, ports 1025 et 8025), tous deux Up et le premier healthy.

Mongo n'est **pas** dans docker-compose.yml : la base est distante (Atlas, replica set obligatoire pour les transactions). Redis non plus. Les deux se lisent dans .env (DATABASE_URL, REDIS_DATABASE_URI).

2.2 Les sujets du courtier

L'auto-crédation de sujets est **désactivée au niveau du cluster** (doctrine A23) : un sujet absent n'est pas créé au vol, il fait échouer la publication. Les deux sujets doivent donc exister explicitement.

```
# Crée booking-events (12 partitions, rétention 7 j) et pose auto_create_topics_enabled=false
./scripts/redpanda-bootstrap.sh

# Le script ne crée PAS messaging-events : à créer une fois, à la main
docker exec yamba-redpanda rpk topic create messaging-events \
  --partitions 12 --replicas 1 --topic-config "retention.ms=604800000"

# Vérification
docker exec yamba-redpanda rpk topic list
```

Attendu : les deux lignes booking-events et messaging-events, chacune avec 12 partitions.

Point de recette à part entière. Le script de démarrage ne crée que booking-events. Sur une machine neuve, si personne ne crée messaging-events, le relais de la messagerie échouera en boucle sur un sujet inconnu — et, au bout de dix tentatives, **parquera** des événements sains. C'est le scénario CRON-RELAIS-6.

2.3 La boîte aux lettres de recette

Dans .env :

```
EMAIL_PROVIDER=smtp
SMTP_HOST=localhost
SMTP_PORT=1025
```

Les emails se lisent ensuite sur `http://localhost:8025` (Mailpit). Aucun email ne part vers l'extérieur. Un email qui n'apparaît pas dans Mailpit n'a pas été envoyé — c'est la preuve d'absence utilisée dans tout ce cahier.

Vider la boîte avant chaque scénario d'email, pour que le verdict soit sans ambiguïté :

```
curl -s -X DELETE http://localhost:8025/api/v1/messages && echo "boîte vidée"
```

Compter les messages reçus :

```
curl -s http://localhost:8025/api/v1/messages | python3 -c "import sys,json; d=json.load(sys.stdin);
print(d['total'], 'message(s)'); [print('-', m['Subject'], '->', [t['Address'] for t in m['To']])
for m in d['messages'][:20]]"
```

2.4 Le jeu d'essai

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-deals.ts
```

C'est la **remise à zéro de recette** : purge et recrée les trajets et réservations des douze comptes de démonstration, mot de passe commun `Yamba-Dev-2026!`, code de livraison `742891` sur toute réservation passée par la remise. Les identifiants créés sont écrits dans `packages/libs/prisma/scripts/seed-output.json` — c'est là qu'on va chercher un identifiant de réservation pour les manipulations de dates.

Remise à zéro des paramètres (les durées de conservation et les seuils d'alerte comptent beaucoup ici) :

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-settings.ts --show # inspecter
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-settings.ts      # remettre aux
défauts
```

Alimentation de l'outbox pour éprouver le relais sans passer par un parcours métier :

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-outbox.ts          # 6 événements
valides
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-outbox.ts --with-poison # + 1 événement
hors contrat
```

Ce script est idempotent : il efface d'abord ses propres lignes (`correlationId = "seed-outbox"`) et n'en touche aucune autre.

2.5 Lire les battements

Chaque tâche laisse, à chaque passage, une trace dans Redis sous la clé `yamba:cron:<service>:<nom>`, au format JSON, avec un TTL de **7 jours** :

```
{ "service": "deal-service", "name": "rating", "ranAt": "...", "durationMs": 120,
  "ok": true, "summary": "2 relance(s), 0 révélation(s)", "error": null, "schedule": "17 * * * *" }
```

Deux façons de les lire.

(a) La page « État des services » du back-office, `http://localhost:3001/status` (profil OPS ou SUPER_ADMIN). Elle appelle `GET /admin/status`, qui agrège les `/health` des six services, **les battements**, l'état de l'outbox (non

publiés, plus ancien non publié, parqués) et les emails des 24 dernières heures. C'est la vue de l'exploitant.

(b) *En ligne de commande*, avec le même code que la page admin. Créer `scripts/recette-battements.ts` :

```
import redis from "../packages/libs/redis";
import { listCronRuns } from "../packages/libs/redis/cron-heartbeat";

(async () => {
  const runs = await listCronRuns(redis);
  runs.sort((a, b) => a.service.localeCompare(b.service) || a.name.localeCompare(b.name));
  console.log(`${runs.length} battement(s)\n`);
  for (const r of runs) {
    const age = Math.round((Date.now() - new Date(r.ranAt).getTime()) / 60000);
    console.log(`${r.ok ? "OK " : "KO "} ${r.service}:${r.name} il y a ${age} min (${r.schedule})`);
    console.log(`${r.summary ?? r.error ?? ""}`);
  }
  process.exit(0);
})();
```

```
npx tsx --env-file=.env scripts/recette-battements.ts
```

Sortie réelle observée à la rédaction de ce cahier (six services démarrés depuis une heure environ) :

```
6 battement(s)

OK auth-service:onboarding-reminder il y a 12 min (0 * * * *) 0 rappel(s)
OK deal-service:expire-bookings il y a 7 min (* / 5 * * * *) 0 expirée(s)
OK deal-service:ops-alerts il y a 7 min (5 * * * *) aucune nouvelle alerte
OK deal-service:payout-bookings il y a 7 min (* / 5 * * * *)
OK deal-service:rating il y a 55 min (17 * * * *) 2 relance(s), 0 révélation(s)
OK message-service:unread-reminder il y a 7 min (* / 5 * * * *) 0 relance(s), 0 échec(s)
```

Lecture de ce relevé. Six battements et pas treize : c'est **normal** une heure après le démarrage. Les tâches nocturnes (03:15 à 03:55) n'ont pas encore eu leur premier passage, et une tâche qui n'a jamais tourné n'a **aucune** clé. Un battement absent signifie donc soit « pas encore passé », soit « ne tourne plus » — c'est l'horaire qui tranche, et c'est précisément pourquoi le chapitre 4 donne un moyen de **forcer** chaque passage.

Trois lectures indispensables :

- `ok: false` avec un `error` : la tâche a tourné et a échoué. C'est le cas le plus facile.
- Battement **plus vieux que deux fois son horaire** : la tâche ne tourne plus. Un `* / 5 * * * *` dont le battement a 20 minutes est en panne.
- Battement **absent au-delà de sa première échéance** : la tâche n'a jamais démarré. C'est le cas de `onboarding-reminder` avant A148.

2.6 Couper une tâche

Toutes les tâches sauf une sont coupables par variable d'environnement, dans le `.env` racine, sur le principe « activée sauf si la variable vaut exactement `false` ».

Tâche	Service	Variable
<code>complete-trips</code>	trip	aucune — non coupable sans redéploiement
<code>expire-bookings</code>	deal	<code>BOOKING_EXPIRY_CRON_ENABLED</code>
<code>payout-bookings</code>	deal	<code>BOOKING_PAYOUT_CRON_ENABLED</code>
<code>ops-alerts</code>	deal	<code>OPS_ALERTS_CRON_ENABLED</code>

Tâche	Service	Variable
ops-digest	deal	OPS_DIGEST_CRON_ENABLED
rating	deal	RATING_CRON_ENABLED
recipient-redaction	deal	RECIPIENT_REDACTION_CRON_ENABLED
outbox-retention (deal)	deal	OUTBOX_RETENTION_CRON_ENABLED
unread-reminder	message	MESSAGING_REMINDER_CRON_ENABLED
conversation-retention	message	MESSAGING_RETENTION_CRON_ENABLED
outbox-retention (message)	message	OUTBOX_RETENTION_CRON_ENABLED
retention	notification	RETENTION_CRON_ENABLED
onboarding-reminder	auth	ONBOARDING_REMINDER_CRON_ENABLED

Et, pour les deux relais et les deux consommateurs :

Composant	Variable
Relais outbox booking	OUTBOX_RELAY_ENABLED
Relais outbox conversation	MESSAGING_RELAY_ENABLED
Les deux consommateurs du notification-service	NOTIFICATION_CONSUMER_ENABLED

À chaque coupure, le service écrit une ligne au démarrage — c'est la preuve que la coupure est prise en compte :

```
Booking expiry cron disabled (BOOKING_EXPIRY_CRON_ENABLED=false)
```

Piège d'exploitation. `nx serve` charge le `.env` racine et **écrase** les variables passées en ligne de commande. Pour forcer un environnement précis, il faut passer par le paquet construit : `sh npx nx build deal-service cd apps/deal-service && BOOKING_EXPIRY_CRON_ENABLED=false node --env-file=../../.env dist/main.js` Node laisse alors l'environnement du processus gagner. C'est la seule méthode fiable pour les scénarios de coupure.

2.7 Forcer un passage : les trois méthodes

C'est le cœur de ce cahier. Il y a exactement trois manières d'obtenir un passage sans attendre l'horaire, et elles ne s'appliquent pas aux mêmes tâches.

Méthode A — appeler la fonction exportée depuis un script. La plupart des tâches délèguent tout leur travail à une fonction ou une méthode exportée, précisément pour être testables. Un script `tsx` lancé à la racine du dépôt résout les alias `@packages/*` (vérifié) et charge le `.env` :

```
npx tsx --env-file=.env scripts/recette-<nom>.ts
```

Deux règles impératives pour ces scripts :

1. **Toujours** `--env-file=.env`. Sans lui, le client Redis part en boucle de reconnexion et le script se fige sans jamais afficher une ligne. Symptôme observé : le script ne rend jamais la main et n'écrit rien.
2. **Toujours terminer par** `process.exit(0)`. Les singletons Redis et Prisma gardent des descripteurs ouverts : sans sortie explicite, le script reste suspendu après avoir tout fait.

Les points d'entrée réellement disponibles :

Tâche	Point d'entrée exporté	Fichier
complete-trips	runCompleteTripsOnce(now?)	apps/trip-service/src/cron/complete-trips.cron.ts
expire-bookings	dealLifecycleService.expireDueBookings(batchSize?)	apps/deal-service/src/routes/deal.routes.ts (instance prête)
payout-bookings	runPayoutPasses(service, logger) ou les trois passes séparément	apps/deal-service/src/cron/payout-bookings.cron.ts
ops-alerts	opsAlertsService.evaluate() / .notifyNewAlerts(redis)	apps/deal-service/src/services/ops-alerts.service.ts
ops-digest	dealSettlementService.collectOpsDigest() + sendOpsDigest(digest, now)	apps/deal-service/src/services/ops-notify.service.ts
rating	dealRatingService.sendRatingReminders() / .revealElapsed()	apps/deal-service/src/services/deal-rating.service.ts
recipient-redaction	makeRecipientRedactionService().runOnce(now?)	apps/deal-service/src/services/recipient-redaction.service.ts
outbox-retention	purgePublishedOutbox(aggregateType, now?)	apps/deal-service/src/cron/outbox-retention.cron.ts
unread-reminder	makeUnreadReminderService().runOnce(now?)	apps/message-service/src/services/unread-reminder.service.ts
conversation-retention	makeConversationRetentionService().purgeOnce(now?)	apps/message-service/src/services/conversation-retention.service.ts
retention (notification)	makeRetentionService().runOnce(now?)	apps/notification-service/src/cron/retention.cron.ts
onboarding-reminder	processOnboardingReminders(now?)	apps/auth-service/src/cron/onboarding-reminder.cron.ts

Quatre instances déjà construites sont exportées par `apps/deal-service/src/routes/deal.routes.ts` : `dealLifecycleService`, `dealSettlementService`, `dealRatingService`, `opsAlertsService`. Les importer évite de reconstruire un fournisseur de paiement à la main.

Méthode B — l'horloge injectée. Presque toutes ces fonctions acceptent un `now` en argument (ou une horloge à la fabrique). C'est la manière **propre** de simuler le futur sans toucher aux données :

```
await makeRecipientRedactionService(() => new Date("2026-12-31T04:00:00Z")).runOnce();
```

Attention : l'horloge injectée décale la **lecture**, mais l'écriture pose ce `now` en base. Un effacement forcé avec une horloge de 2027 écrira `recipientRedactedAt = 2027-....`. C'est acceptable en recette, jamais en production.

Méthode C — rendre un élément éligible en base. Quand la tâche sélectionne sur une date passée, la manière la plus fidèle est de **reculer la date en base** puis de laisser le cron passer tout seul (5 minutes au pire) ou de forcer le passage par la méthode A. C'est la méthode à privilégier pour les tâches à tick court, parce qu'elle éprouve **aussi** l'ordonnanceur, pas seulement la règle.

Chaque chapitre du § 4 indique **le champ exact du modèle exact** à modifier. Le patron d'un tel script :

```
import prisma from "../../packages/libs/prisma";

const ID = process.argv[2]; // identifiant de la réservation
const daysAgo = (n: number) => new Date(Date.now() - n * 86_400_000);

(async () => {
  const before = await prisma.booking.findUnique({ where: { id: ID } });
  console.log("avant :", before?.status, before?.expiresAt);
  await prisma.booking.update({ where: { id: ID }, data: { expiresAt: daysAgo(2) } });
  const after = await prisma.booking.findUnique({ where: { id: ID } });
  console.log("après :", after?.status, after?.expiresAt);
  process.exit(0);
})();
```

Piège Mongo, à connaître avant toute manipulation. Sur MongoDB, un champ **jamais écrit** est **absent**, ce qui n'est pas la même chose que `null`. Un filtre `{ champ: null }` **ne trouve pas** les documents où le champ est absent. C'est pour cela que le code écrit partout `OR: [{ champ: null }, { champ: { isSet: false } }]`. Conséquence pour le testeur : une fixture qui **pose** explicitement `verificationReminderSentAt: null` ne prouve rien sur le comportement réel avec un document où le champ n'a jamais existé. Pour reproduire le vrai cas, il faut **supprimer** le champ (`{ unset: true }` côté Prisma Mongo, ou repartir du seed).

2.8 Lire l'état de l'outbox

Beaucoup de vérifications de ce cahier passent par la collection `OutboxEvent`. Un script de lecture unique, `scripts/recette-outbox.ts` :

```
import prisma from "../../packages/libs/prisma";

(async () => {
  const absent = { OR: [{ publishedAt: null }, { publishedAt: { isSet: false } } ] } as never;
  const [total, nonPublies, parques, plusAncien, parType] = await Promise.all([
    prisma.outboxEvent.count(),
    prisma.outboxEvent.count({ where: absent }),
    prisma.outboxEvent.count({ where: { ...(absent as object), attempts: { gte: 10 } } as never }),
    prisma.outboxEvent.findFirst({ where: absent, orderBy: { occurredAt: "asc" }, select: { id:
true, eventType: true, occurredAt: true, attempts: true, lastError: true } }),
    prisma.outboxEvent.groupBy({ by: ["aggregateType"], _count: true }),
  ]);
  console.log({ total, nonPublies, parques });
  console.log("plus ancien non publié :", plusAncien);
  console.log("par type d'agrégat :", parType);
  process.exit(0);
})();
```

Les mêmes chiffres apparaissent sur la page « État des services » du back-office, bloc **Outbox** : `unpublished`, `oldestUnpublishedAt`, `parked`, `parkedThreshold`.

2.9 Lire et consommer le courtier

```
# Ce qui existe
docker exec yamba-redpanda rpk topic list
docker exec yamba-redpanda rpk topic describe booking-events

# Lire les N derniers messages sans consommer d'offset de groupe
docker exec yamba-redpanda rpk topic consume booking-events -n 5 -o -5
docker exec yamba-redpanda rpk topic consume messaging-events -n 5 -o -5

# Suivre en direct pendant un scénario
docker exec yamba-redpanda rpk topic consume booking-events -f '%k %h %v\n'

# L'état des groupes de consommation (retard = lag)
docker exec yamba-redpanda rpk group list
docker exec yamba-redpanda rpk group describe notification-service
docker exec yamba-redpanda rpk group describe messaging-notifications
```

Les deux groupes s'appellent exactement `notification-service` et `messaging-notifications` (packages/libs/messaging/src/consumer-groups.ts). Ces noms ne doivent **jamais** changer : les renommer, c'est perdre les offsets et tout retraiter.

3. Conventions

3.1 Numérotation

CRON-<NOM>-<n> où <NOM> désigne la tâche ou le composant :

Préfixe	Objet
CRON-TRAJETS-*	Complétion des trajets (trip)
CRON-EXPIRE-*	Expiration des demandes à 24 h (deal)
CRON-PAYOUT-*	Versement J+4, rejeu, rappel J+3 (deal)
CRON-ALERTES-*	Alertes de seuil (deal)
CRON-NOTATION-*	Relances et révélation des avis (deal)
CRON-DIGEST-*	Récapitulatif quotidien (deal)
CRON-RELANCE-*	Relance des messages non lus (message)
CRON-PURGEFIL-*	Purge des conversations (message)
CRON-PURGEOUT-*	Purge des événements publiés (deal, message)
CRON-DESTINATAIRE-*	Effacement du tiers destinataire (deal)
CRON-CONSERV-*	Conservation générale (notification)
CRON-ONBOARD-*	Rappels d'inscription Voyageur (auth)
CRON-RELAIS-*	Relais d'événements
CRON-CONSO-*	Consommateurs
CRON-BATT-*	Battements et moniteur externe
CRON-SEC-*	Sécurité et conformité

3.2 Verdicts

Verdict	Signification
Conforme	Le résultat observé correspond exactement au résultat attendu
Non conforme	Écart entre l'observé et l'attendu — une anomalie est ouverte
Non joué	Prérequis indisponible (courtier arrêté, jeu d'essai absent) — à rejouer, jamais à classer conforme
Sans objet	Le scénario ne s'applique pas à cet environnement (par exemple un moniteur externe non déclaré)

3.3 Gravité des anomalies

Gravité	Critère	Exemple type
Bloquante	De l'argent ne part pas, ou part deux fois ; une donnée personnelle survit à sa durée de conservation ; un secret sort dans un payload	Le versement J+4 ne s'exécute pas ; un code de livraison dans un événement
Majeure	Une tâche ne tourne plus sans que rien ne l'indique ; un doublon d'email ou de notification ; un événement sain parqué	Le relais parqué des événements parce qu'un sujet manque
Mineure	Compteur, libellé, résumé de battement inexact ; écart documentaire	Un battement dont le <code>summary</code> ne compte pas ce qu'il annonce
Cosmétique	Formulation d'un email, mise en forme	Une date au mauvais format dans le récapitulatif

3.4 La fiche type d'une tâche

Chaque chapitre du § 4 ouvre sur cette fiche, remplie à partir du code, jamais de mémoire :

Rubrique	Ce qu'elle contient
Fichier	Le chemin exact du cron
Horaire réel	L'expression cron telle qu'elle est dans le code, avec sa traduction
Interrupteur	La variable d'environnement, ou « aucun »
Nom du battement	La clé Redis <code>yamba:cron:<service>:<nom></code>
Garde de chevauchement	Le mécanisme qui empêche deux passages simultanés
Service appelé	Le fichier et la méthode où se trouve le travail réel
Rendre un élément éligible	Le champ exact, du modèle exact, à modifier
Comment déclencher	La commande à lancer
Preuve attendue	L'effet mesurable : base, email, événement, battement

3.5 Les treize tâches, vue d'ensemble

#	Service	Tâche	Horaire	Nature
1	trip	<code>complete-trips</code>	<code>15 3 * * *</code>	Nocturne
2	deal	<code>expire-bookings</code>	<code>* / 5 * * * *</code>	Tick court
3	deal	<code>payout-bookings</code>	<code>* / 5 * * * *</code>	Tick court, argent
4	deal	<code>ops-alerts</code>	<code>5 * * * *</code>	Horaire
5	deal	<code>rating</code>	<code>17 * * * *</code>	Horaire
6	deal	<code>ops-digest</code>	<code>0 8 * * *</code>	Quotidienne
7	message	<code>unread-reminder</code>	<code>* / 5 * * * *</code>	Tick court

#	Service	Tâche	Horaire	Nature
8	message	conversation-retention	30 3 * * *	Nocturne, suppression
9	deal	outbox-retention	55 3 * * *	Nocturne, suppression
10	message	outbox-retention	55 3 * * *	Nocturne, suppression
11	deal	recipient-redaction	40 3 * * *	Nocturne, RGPD
12	notification	retention	50 3 * * *	Nocturne, suppression
13	auth	onboarding-reminder	0 * * * *	Horaire

Les nocturnes sont décalées de cinq minutes les unes des autres (03:15, 03:30, 03:40, 03:50, 03:55) pour ne pas se disputer la base au même instant. Ce décalage fait partie de ce qu'on vérifie : deux tâches lourdes au même instant sur un cluster Atlas partagé, c'est une source de lenteur inexplicable.

4. Les treize tâches, une par une

4.1 Complétion des trajets — complete-trips

Rubrique	Valeur
Fichier	apps/trip-service/src/cron/complete-trips.cron.ts
Horaire réel	15 3 * * * — tous les jours à 03:15, heure serveur
Interrupteur	aucun (DIV-2) — la seule façon de la couper est de retirer l'appel dans apps/trip-service/src/main.ts
Nom du battement	yamba:cron:trip-service:complete-trips
Garde de chevauchement	let started = false au niveau du module : startCompleteTripsCron() est idempotent, mais il n'y a pas de garde sur le tick — un passage qui déborde de 24 h chevaucherait le suivant
Service appelé	runCompleteTripsOnce(now) dans le même fichier ; la décision passe par canPerform de apps/trip-service/src/services/trip-state-machine.ts

Ce qu'elle fait. Elle cherche les trajets PUBLISHED ou PAUSED, non supprimés, dont l'arrivée (arrivalAt, à défaut departureAt) est passée depuis plus de **24 heures** de grâce. Pour chacun, elle demande à la machine à états si l'action complete est permise, en lui donnant deux informations lues en base : hasActiveBookings(trip.id) et hasBookingsInProgress(trip.id). Si oui, le trajet passe COMPLETED et la page Voyageur perd un trajet publié (getCarrierStatDeltas).

Rendre un trajet éligible. Modèle Trip, champ arrivalAt (ou departureAt si arrivalAt est absent), à reculer de plus de 24 h.

```
// scripts/recette-trip-eligible.ts
import prisma from "../packages/libs/prisma";
const ID = process.argv[2];
(async () => {
  const t = await prisma.trip.update({
    where: { id: ID },
    data: { status: "PUBLISHED", isDeleted: false,
      departureAt: new Date(Date.now() - 4 * 86_400_000),
      arrivalAt: new Date(Date.now() - 3 * 86_400_000) },
  });
  console.log("trajet rendu éligible :", t.id, t.status, t.arrivalAt);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-complete-trips.ts
import { runCompleteTripsOnce } from "../apps/trip-service/src/cron/complete-trips.cron";
(async () => { console.log(await runCompleteTripsOnce()); process.exit(0); })();
```

```
npx tsx --env-file=.env scripts/recette-complete-trips.ts
```

CRON-TRAJETS-1 — Le trajet terminé passe COMPLETED

Objectif	Vérifier le cas nominal : un trajet dont le voyage est fini depuis plus de 24 h et qui n'a plus de deal actif est terminé
Prérequis	Jeu d'essai rejoué ; un trajet PUBLISHED sans réservation en cours (relever son identifiant dans seed-output.json)

Étapes

1. Rendre le trajet éligible avec `recette-trip-eligible.ts`.
2. Noter son statut avant : il doit être PUBLISHED.
3. Noter le compteur `totalTripsPublished` de la `CarrierPage` du Voyageur.
4. Lancer `recette-complete-trips.ts`.

Résultat attendu

- Le script affiche `{ scanned: n, completed: ≥ 1, skipped: ... }` et la ligne de journal `[complete-trips] scanned=... completed=... skipped=...`.
- En base, le `Trip` est passé à COMPLETED.
- `CarrierPage.totalTripsPublished` a **diminué de 1** : le trajet sort du vivier public.
- Le trajet n'apparaît plus dans la recherche côté membre.

Verdict : ☐ conforme ☐ non conforme

CRON-TRAJETS-2 — Le trajet avec un deal en cours n'est PAS terminé

Objectif	Vérifier le cas où la tâche ne doit pas agir : un trajet dont un deal est encore en cours reste ouvert, même si le voyage est fini depuis longtemps
Prérequis	Un trajet avec au moins une réservation ACCEPTED ou PICKED_UP

Étapes

1. Reculer `arrivalAt` de ce trajet de 4 jours.
2. Vérifier que la réservation est bien dans un statut logistique non terminal.
3. Lancer `recette-complete-trips.ts`.

Résultat attendu

- Le compteur `skipped` du script a augmenté ; `completed` **n'inclut pas** ce trajet.
- Le `Trip` est resté PUBLISHED (ou PAUSED).
- Le refus vient de la machine à états (`canPerform` a répondu non), pas d'un `if` dans le cron : c'est la garantie que la règle ne diverge pas entre l'écran et la tâche.

Verdict : ☐ conforme ☐ non conforme

CRON-TRAJETS-3 — Non-régression : le litige ne bloque pas la complétion

Objectif	Vérifier la décision A20 : un deal <code>DISPUTED</code> a un voyage terminé ; il gèle le versement, pas le trajet
Prérequis	Un trajet dont l'unique réservation est en <code>DISPUTED</code>

Étapes — reculer `arrivalAt` de 4 jours, lancer la tâche.

Résultat attendu — le trajet passe `COMPLETED`. Un trajet éternellement ouvert à cause d'un litige serait une régression : le litige se règle côté argent, pas côté logistique.

Verdict : ☐ conforme ☐ non conforme

CRON-TRAJETS-4 — Non-régression : un trajet en échec ne bloque pas la journée

Objectif	Vérifier que l'erreur sur un trajet est absorbée et que les suivants sont traités

Étapes

1. Rendre **trois** trajets éligibles.
2. Provoquer un échec sur l'un d'eux — le plus simple : supprimer sa `CarrierPage` (`prisma.carrierPage.delete`) pour que la mise à jour des compteurs soit sans cible, ou passer son `userId` sur un identifiant inexistant.
3. Lancer la tâche.

Résultat attendu — les deux autres trajets sont bien `COMPLETED` ; le troisième est compté dans `skipped` et une ligne `[complete-trips] Failed to complete trip <id>` apparaît. La passe **entière** ne s'arrête pas sur un cas.

Verdict : ☐ conforme ☐ non conforme

4.2 Expiration des demandes à 24 h — `expire-bookings`

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/expire-bookings.cron.ts</code>
Horaires réel	<code>* / 5 * * * *</code> — toutes les cinq minutes
Interrupteur	<code>BOOKING_EXPIRY_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:expire-bookings</code>
Garde de chevauchement	Drapeau <code>running</code> : un tick encore en vol fait sauter le suivant ; le retard se résorbe au tick d'après
Service appelé	<code>expireDueBookings(batchSize = 50)</code> dans <code>apps/deal-service/src/services/deal-lifecycle.service.ts</code>

Ce qu'elle fait. Elle cherche les réservations `PENDING`, non supprimées, dont `expiresAt` est strictement dépassé (`lt: now`), par fournées de **50**, triées par `expiresAt` croissant. Pour chacune : la machine à états valide la transition `expire` par l'acteur `SYSTEM` (garde `onlyIfExpired`), l'empreinte de paiement est libérée (`provider.cancel`, au mieux — elle expirerait seule de toute façon), puis une **transaction** unique écrit le statut, restitue les kilos au trajet et pose **deux événements d'outbox**.

Point important de conception : la machine considère déjà un `PENDING` périmé **comme expiré** avant même le passage du cron. La tâche ne décide donc rien : elle **matérialise** l'état et libère l'argent et les kilos.

Effets exacts d'un passage sur une réservation :

Effet	Détail
Statut	PENDING → EXPIRED
Champs	closedAt = now, closedBy = "SYSTEM"
Kilos	Trip.reservedKg décrémenté de booking.pricing.weightKg, garde reservedKg >= kg
Empreinte	provider.cancel(paymentIntentId, "requested_by_customer") — best effort
Événements	booking.expired (avec closedAt) et booking.refund_issued (avec amountCents = pricing.totalShipperCents)

Rendre une demande éligible. Modèle `Booking`, champ `expiresAt`, à reculer dans le passé, avec `status: "PENDING"`.

```
// scripts/recette-expire-eligible.ts
import prisma from "../packages/libs/prisma";
const ID = process.argv[2];
(async () => {
  const b = await prisma.booking.update({
    where: { id: ID },
    data: { status: "PENDING", isDeleted: false, expiresAt: new Date(Date.now() - 3_600_000) },
  });
  console.log("demande périmée :", b.id, b.status, b.expiresAt);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-expire.ts
import { dealLifecycleService } from "../apps/deal-service/src/routes/deal.routes";
(async () => { console.log(await dealLifecycleService.expireDueBookings(), "expirée(s)");
  process.exit(0); })();
```

Variante recommandée : **ne rien forcer** et attendre au plus cinq minutes. Cela éprouve en plus l'ordonnanceur et le battement.

CRON-EXPIRE-1 — La demande dépassée expire et libère tout

Objectif	Cas nominal complet : statut, kilos, empreinte, deux événements
Prérequis	Une réservation PENDING du jeu d'essai; relever <code>tripId</code> , <code>pricing.weightKg</code> , <code>pricing.totalShipperCents</code>

Étapes

1. Relever `Trip.reservedKg` du trajet porteur.
2. Rendre la demande éligible (`expiresAt` il y a une heure).
3. Attendre le tick de cinq minutes **ou** lancer `recette-expire.ts`.

Résultat attendu

- La réservation est `EXPIRED`, avec `closedAt` renseigné et `closedBy` = "SYSTEM".
- `Trip.reservedKg` a **diminué exactement** de `pricing.weightKg`.
- Deux lignes neuves dans `OutboxEvent` pour cet `aggregateId` : `booking.expired` et `booking.refund_issued`. Le second porte `amountCents` = `pricing.totalShipperCents`.
- Le journal du deal-service affiche `Expired overdue booking requests (24h window)` avec le compte.

- Le battement `deal-service:expire-bookings` est à jour, `ok: true`, `summary` = « 1 expirée(s) ».
- Côté membre (cahier 01) : l'Expéditeur reçoit la notification et l'email d'expiration ; le Voyageur ne voit plus la demande.

Verdict : ☐ conforme ☐ non conforme

CRON-EXPIRE-2 — La demande non dépassée n'est PAS touchée

Objectif	Vérifier la borne : le filtre est <code>lt</code> (strictement dépassé), pas <code>lte</code>
-----------------	---

Étapes

1. Poser `expiresAt` **dans le futur proche** (par exemple `now + 10 min`) sur une réservation `PENDING`.
2. Lancer la tâche.

Résultat attendu — la réservation reste `PENDING`, aucun événement, aucun kilo libéré, le compteur renvoyé par la tâche ne l'inclut pas.

Variante à jouer aussi : une réservation `ACCEPTED` avec un `expiresAt` passé. Elle ne doit **pas** expirer : le filtre porte sur `status: "PENDING"`. Une acceptation ne se défait pas parce qu'une échéance de demande est derrière nous.

Verdict : ☐ conforme ☐ non conforme

CRON-EXPIRE-3 — Non-régression : pas de double expiration, pas de double restitution de kilos

Objectif	Prouver que rejouer la tâche sur la même réservation ne restitue pas les kilos deux fois
-----------------	--

Étapes

1. Jouer CRON-EXPIRE-1 jusqu'au bout ; relever `Trip.reservedKg`.
2. Relancer **immédiatement** `recette-expire.ts`, deux fois de suite.

Résultat attendu

- La deuxième et la troisième exécution renvoient `0`.
- `Trip.reservedKg` est **inchangé** par rapport au relevé de l'étape 1.
- Aucun nouvel événement `booking.expired` n'apparaît.

La protection vient de deux endroits : le filtre `status: "PENDING"` ne retrouve plus la réservation, et l'écriture est conditionnelle (`updateMany` sur `{ id, status: "PENDING" }`) — même si deux instances lisaient la même ligne, une seule écrirait.

Verdict : ☐ conforme ☐ non conforme

CRON-EXPIRE-4 — Non-régression : la garde de chevauchement

Objectif	Vérifier qu'un passage long ne se fait pas doubler par le tick suivant
-----------------	--

Étapes

1. Rendre éligibles **plus de 50 réservations** (la fournie est de 50) — script en boucle sur les réservations `PENDING` du jeu d'essai.

2. Laisser tourner deux ticks (dix minutes) sans forcer.

Résultat attendu

- Le premier tick traite 50 réservations, le second le reste : le retard se résorbe, il ne s'accumule pas.
- Aucune réservation n'est traitée deux fois (à vérifier : pas de `closedAt` écrasé, pas de double décrémentation de `reservedKg`).
- Un seul battement par tick, jamais deux au même horodatage.

Verdict : ☐ conforme ☐ non conforme

CRON-EXPIRE-5 — La coupure par variable d'environnement

Étapes

```
npx nx build deal-service
cd apps/deal-service && BOOKING_EXPIRY_CRON_ENABLED=false node --env-file=../../.env dist/main.js
```

Résultat attendu

- Au démarrage : Booking expiry cron disabled (`BOOKING_EXPIRY_CRON_ENABLED=false`).
- Une demande rendue éligible **reste** `PENDING` après quinze minutes.
- Le battement `deal-service:expire-bookings` **cesse de vieillir** : au bout de sept jours il disparaîtrait (TTL). C'est exactement le signal que le moniteur externe doit relever (§ 7).

Verdict : ☐ conforme ☐ non conforme

4.3 Versement à J+4, rejeu et rappel — `payout-bookings`

C'est la tâche qui déplace de l'argent. Toute anomalie ici est **bloquante** par défaut.

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/payout-bookings.cron.ts</code>
Horaire réel	<code>* / 5 * * * *</code>
Interrupteur	<code>BOOKING_PAYOUT_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:payout-bookings</code> (sans résumé : <code>runPayoutPasses</code> ne renvoie rien)
Garde de chevauchement	Drapeau <code>running</code> , comme l'expiration
Service appelé	<code>apps/deal-service/src/services/deal-settlement.service.ts</code>

Trois passes, dans cet ordre, à chaque tick (`runPayoutPasses`) :

#	Méthode	Sélection	Effet
1	<code>autoCompleteDue(50)</code>	<code>status: "DELIVERED"</code> , <code>payoutDueAt <= now</code>	→ <code>COMPLETED</code> par <code>SYSTEM</code> , puis exécution du transfert
2	<code>retryFailedPayouts(50)</code>	<code>status ∈ {COMPLETED, CANCELLED}</code> , <code>payoutStatus ∈ {PENDING, FAILED}</code> , échéance de rejeu atteinte	Nouveau transfert
3	<code>sendVerificationReminders(50)</code>	<code>status: "DELIVERED"</code> , <code>payoutDueAt entre now et now+24 h</code> , rappel jamais envoyé	Événement <code>booking.verification_reminder</code>

Champs écrits à la complétion (`completeBooking`): `status = COMPLETED`, `completedAt`, `completedBy = "SYSTEM"`, `payoutStatus = "PENDING"`, `payoutAmountCents = pricing.transportCents`, `payoutFailureReason = null`, `ratingWindowEndsAt = now + rating.windowDays` (défaut 14 jours), `ratingRemindersSent = 0`. Événement `booking.completed`.

Champs écrits au transfert réussi (`executePayout`): `payoutStatus = "SENT"`, `transferId`, `payoutSentAt`, `payoutAttempts + 1`, `payoutFailureReason = null`, `payoutLastAttemptAt`, `payoutNextRetryAt = null`. Événement `booking.payout_sent` avec `reason = "DELIVERY"` (ou `"LATE_CANCELLATION"` pour une annulation tardive). Clé d'idempotence fournisseur: `payoutIdempotencyKey ?? "payout:<bookingId>"`.

Champs écrits à l'échec (`markPayoutFailed`): `payoutStatus = "FAILED"`, `payoutFailureReason` (`CARRIER_ACCOUNT_NOT_READY` ou `PROVIDER_ERROR:<message>`), `payoutAttempts`, `payoutLastAttemptAt`, `payoutNextRetryAt`.

Espacement des rejeux (`payoutRetryDelayMs`, `apps/deal-service/src/services/admin-finance.rules.ts`):

Tentatives déjà faites	Prochain essai dans
< 6	5 minutes
< 12	30 minutes
< 24	2 heures
≥ 24	24 heures

Il n'y a **aucun plafond** de tentatives (DIV-3) : un versement en échec est retenté indéfiniment, de plus en plus rarement, jusqu'à ce que le compte Voyageur soit prêt ou qu'un administrateur intervienne.

Rendre une réservation éligible. Modèle `Booking` :

Passe visée	Champs à poser
Versement J+4	<code>status: "DELIVERED"</code> , <code>payoutDueAt</code> dans le passé
Rejeu	<code>status: "COMPLETED"</code> , <code>payoutStatus: "FAILED"</code> , <code>payoutNextRetryAt</code> dans le passé (ou absent)
Rappel J+3	<code>status: "DELIVERED"</code> , <code>payoutDueAt</code> entre now et now + 24 h , <code>verificationReminderSentAt</code> absent

```
// scripts/recette-payout-eligible.ts - usage : ... <bookingId> <due|retry|reminder>
import prisma from "../packages/libs/prisma";
const [ID, MODE] = process.argv.slice(2);
const h = (n: number) => new Date(Date.now() + n * 3_600_000);
(async () => {
  const data =
    MODE === "due"      ? { status: "DELIVERED" as const, payoutDueAt: h(-1) }
  : MODE === "retry"    ? { status: "COMPLETED" as const, payoutStatus: "FAILED" as const,
                          payoutFailureReason: "CARRIER_ACCOUNT_NOT_READY",
                          payoutNextRetryAt: h(-1), payoutAttempts: 1 }
  :                      { status: "DELIVERED" as const, payoutDueAt: h(6) };
  const b = await prisma.booking.update({ where: { id: ID }, data });
  console.log(MODE, "-", b.status, b.payoutStatus, b.payoutDueAt, b.payoutNextRetryAt);
  process.exit(0);
})();
```

Pour le mode `reminder`, il faut en plus que `verificationReminderSentAt` soit **absent** (pas `null` : voir le piège Mongo du § 2.7). Le plus sûr est de repartir d'une réservation fraîche du jeu d'essai.

Déclencher.

```
// scripts/recette-payout.ts
import pino from "pino";
import { dealSettlementService } from "../apps/deal-service/src/routes/deal.routes";
import { runPayoutPasses } from "../apps/deal-service/src/cron/payout-bookings.cron";
(async () => {
  const logger = pino({ level: "info" });
  const which = process.argv[2] ?? "all";
  if (which === "complete") console.log("complétées :", await dealSettlementService.autoCompleteDue());
  else if (which === "retry") console.log("rejouées :", await dealSettlementService.retryFailedPayouts());
  else if (which === "remind") console.log("rappels :", await dealSettlementService.sendVerificationReminders());
  else await runPayoutPasses(dealSettlementService, logger);
  process.exit(0);
})();
```

Fournisseur de paiement. En recette locale, laisser `STRIPE_SECRET_KEY` vide : la fabrique renvoie le `FakePaymentProvider`, qui adopte les intentions `pi_fake_seed_*` du jeu d'essai et rend des transferts factices. Le Fake est **refusé en production** : ce n'est pas un risque de fuite vers le vrai Stripe.

CRON-PAYOUT-1 — Le versement à échéance part

Objectif	Cas nominal : une remise dont le délai de vérification est écoulé se termine et le Voyageur est payé
Prérequis	Une réservation <code>DELIVERED</code> du jeu d'essai, avec un <code>chargeId</code> et un <code>paymentIntentId</code>

Étapes

1. `recette-payout-eligible.ts` `<id>` `due`.
2. Relever `payoutStatus`, `transferId`, `payoutAttempts` avant.
3. Lancer `recette-payout.ts` `complete`.

Résultat attendu

- Statut `DELIVERED` → **COMPLETED**, `completedBy` = `"SYSTEM"`, `completedAt` renseigné.
- `payoutAmountCents` **égal à** `pricing.transportCents` — jamais recalculé depuis le trajet, c'est l'instantané figé.
- `ratingWindowEndsAt` = `completedAt` + 14 jours, `ratingRemindersSent` = 0.
- `payoutStatus` = `"SENT"`, `transferId` renseigné, `payoutSentAt` renseigné, `payoutAttempts` incrémenté de 1, `payoutNextRetryAt` remis à `null`.
- Deux événements dans `OutboxEvent` : `booking.completed` puis `booking.payout_sent` (avec `reason: "DELIVERY"`).
- Après passage du relais : deux notifications in-app (les **deux** parties pour `completed`, le **Voyageur seul** pour `payout_sent`) et les emails correspondants dans Mailpit.

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-2 — La remise non échue n'est PAS versée

Objectif	Vérifier la borne <code>payoutDueAt</code> <code><= now</code> et la garde <code>onlyIfPayoutDue</code> de la machine
-----------------	--

Étapes — poser `status: "DELIVERED"` et `payoutDueAt = now + 2 jours`, puis lancer `recette-payout.ts complete`.

Résultat attendu

- La réservation reste `DELIVERED`.
- Aucun `transferId`, aucun `payoutStatus`.
- Aucun événement `booking.completed`.

Variante obligatoire : la même chose avec `status: "DISPUTED"` et `payoutDueAt` **passé**. Le versement ne doit **pas** partir : un litige gèle l'argent (`payoutStatus: "FROZEN"`, invariant INV-5). C'est le scénario le plus important de ce chapitre — un versement qui part pendant un litige est irrattrapable.

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-3 — Le rappel de vérification à J+3

Objectif	L'Expéditeur est prévenu 24 h avant la fin de son délai de vérification
Prérequis	Une réservation <code>DELIVERED</code> fraîche (champ <code>verificationReminderSentAt</code> absent)

Étapes

1. Poser `payoutDueAt = now + 6 h` (dans la fenêtre `]now ; now+24 h]`).
2. Vider Mailpit.
3. Lancer `recette-payout.ts remind`.

Résultat attendu

- Le script renvoie `1`.
- `verificationReminderSentAt` est renseigné en base.
- Un événement `booking.verification_reminder` est dans l'outbox, avec `payoutDueAt` au format ISO.
- Après le relais : une notification in-app pour **l'Expéditeur seul** (matrice A15) et un email dans Mailpit.

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-4 — Non-régression : jamais deux rappels

Objectif	Vérifier la garde <code>verificationReminderSentAt</code> — un rappel envoyé deux fois est une faute visible par le membre
-----------------	--

Étapes — après CRON-PAYOUT-3, relancer `recette-payout.ts remind` trois fois de suite.

Résultat attendu

- Chaque relance renvoie `0`.
- Aucun nouvel événement `booking.verification_reminder`.
- Mailpit ne contient toujours qu'**un seul** email de rappel.

La garde est double : le filtre `OR: [{ verificationReminderSentAt: null }, { ... isSet: false }]` à la lecture, **et** la même condition réappliquée dans la transaction d'écriture (message de conflit « Reminder already sent. »). C'est cette seconde garde qui protège de deux instances concurrentes.

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-5 — Le rejeu espacé d'un versement en échec

Objectif	Vérifier que l'échec est retenté, et que le rythme des rejeux ralentit
----------	--

Étapes

1. `recette-payout-eligible.ts` `<id> retry` (`pose payoutStatus: "FAILED"`, `payoutAttempts: 1`, `payoutNextRetryAt` dans le passé).
2. Lancer `recette-payout.ts retry`. Relever `payoutStatus`, `payoutAttempts`, `payoutNextRetryAt`.
3. Rejouer avec `payoutAttempts` forcé à 6, puis 12, puis 24, en remettant `payoutStatus: "FAILED"` et `payoutNextRetryAt` dans le passé à chaque fois, et en provoquant un nouvel échec (par exemple en vidant `chargeId`, ou en rendant le compte Voyageur non prêt).

Résultat attendu

- Au premier rejeu réussi : `payoutStatus = "SENT"`, `payoutNextRetryAt = null`.
- À chaque échec, `payoutNextRetryAt` est repoussé selon la grille : **5 min** sous 6 tentatives, **30 min** sous 12, **2 h** sous 24, **24 h** au-delà.
- Une réservation dont `payoutNextRetryAt` est **dans le futur** n'est **pas** reprise par la passe de rejeu (c'est le cas « ne doit pas agir » de cette passe).
- Une réservation dont `payoutNextRetryAt` est **absent** l'est (filtre `isSet: false`).

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-6 — Non-régression : jamais deux transferts pour la même réservation

Objectif	La garantie la plus importante du système : un double versement
----------	---

Étapes

1. Jouer CRON-PAYOUT-1 (versement parti, `payoutStatus = "SENT"`).
2. Relever `transferId` et `payoutAmountCents`.
3. Lancer `recette-payout.ts all` cinq fois de suite, sans rien modifier.

Résultat attendu

- Aucune exécution ne renvoie un versement supplémentaire.
- `transferId` **inchangé**, `payoutSentAt` **inchangé**, `payoutAttempts` **inchangé**.
- Un seul événement `booking.payout_sent` dans l'outbox pour cet agrégat.

Deux verrous protègent ce point : la garde d'écriture `where: { payoutStatus: { in: ["PENDING", "FAILED"] } }` (un `SENT` n'est jamais réécrit) et la **clé d'idempotence** transmise au fournisseur (`payout:<bookingId>`), qui ferait rejeter un doublon même côté Stripe.

Verdict : ☐ conforme ☐ non conforme

CRON-PAYOUT-7 — L'ordre des trois passes

Objectif	Vérifier que la complétion précède le rejeu, qui précède les rappels
----------	--

Étapes — préparer trois réservations, une pour chaque passe, puis lancer `recette-payout.ts all` avec le journal en `info`.

Résultat attendu — dans le journal, dans cet ordre : `Auto-completed deals past their verification window (D+4)` , puis `Retried carrier payouts sent` , puis `Verification reminders (D+3) emitted` . L'ordre n'est pas cosmétique : une réservation complétée au premier passage doit pouvoir être rejouée au même tick si son transfert a échoué.

Verdict : ☐ conforme ☐ non conforme

4.4 Alertes de seuil — `ops-alerts`

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/ops-alerts.cron.ts</code>
Horaire réel	<code>5 * * * *</code> — toutes les heures, à la cinquième minute
Interrupteur	<code>OPS_ALERTS_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:ops-alerts</code>
Garde de chevauchement	aucune — pas de drapeau <code>running</code> sur ce cron (le travail est court : neuf comptages)
Service appelé	<code>notifyNewAlerts(store)</code> dans <code>apps/deal-service/src/services/ops-alerts.service.ts</code> , règles pures dans <code>ops-alerts.rules.ts</code>

Ce qu'elle fait. Elle prend un instantané de neuf indicateurs (`collectOpsSnapshot`), applique les neuf règles avec les seuils lus dans les paramètres, et pour chaque alerte **apparue pour la première fois aujourd'hui**, envoie **un** email au support. Le dédoublemage est une clé Redis posée en `NX` :

```
yamba:alerts:sent:<RÈGLE>:<AAAA-MM-JJ>      TTL 172 800 s (48 h)
```

La date est en **UTC** (`now.toISOString().slice(0, 10)`). L'accueil du back-office, lui, recalcule les mêmes règles **à chaque lecture** : la page est toujours à jour, le cron ne sert qu'à pousser l'email.

Les neuf règles :

rule	Gravité	Condition	Seuil paramétrable
<code>PAYOUT_FAILED_48H</code>	critique	Au moins un versement en échec depuis plus de N heures	<code>alerts.payoutFailedHours</code> (48)
<code>DISPUTE_UNDECIDED_72H</code>	critique	Litige décidable et non décidé depuis plus de N heures	<code>alerts.disputeUndecidedHours</code> (72)
<code>RETENTION_HELD_7D</code>	avertissement	Retenue d'annulation non arbitrée depuis N jours	<code>alerts.retentionHeldDays</code> (7)
<code>REVERSAL_OPEN_48H</code>	avertissement	Renversement de transfert ouvert depuis N heures	<code>alerts.reversalOpenHours</code> (48)
<code>OUTBOX_PARKED</code>	critique	Au moins un événement parqué	<code>alerts.outboxParkedAttempts</code> (10)
<code>OUTBOX_LAGGING_15MIN</code>	critique	Le plus ancien non publié a plus de N minutes	<code>alerts.outboxLagMinutes</code> (15)
<code>EMAILS_FAILED_24H</code>	avertissement	Au moins un email en échec dans la fenêtre	<code>alerts.emailsFailedWindowHours</code> (24)
<code>NO_TRIP_PUBLISHED_7D</code>	avertissement	Aucun trajet publié depuis N jours	<code>alerts.noTripPublishedDays</code> (7)

rule	Gravité	Condition	Seuil paramétrable
ACCEPTANCE_RATE_LOW_7D	avertissement	Taux d'acceptation sous N % sur la fenêtre, si assez de demandes	alerts.acceptanceRateMinPct (30), ... WindowDays (7), ...MinRequests (5)

Rendre une alerte vraie. Chaque règle a son levier :

Règle	Ce qu'il faut fabriquer
PAYOUT_FAILED_48H	Un Booking COMPLETED avec payoutStatus: "FAILED" et completedAt il y a plus de 48 h
OUTBOX_PARKED	seed-outbox.ts --with-poison puis laisser le relais atteindre 10 tentatives (≈ 10 ticks, une dizaine de secondes), ou poser attempts: 10 directement
OUTBOX_LAGGING_15MIN	Couper le relais (OUTBOX_RELAY_ENABLED=false), poser une ligne d'outbox non publiée avec occurredAt il y a 20 min
EMAILS_FAILED_24H	Un EmailDelivery avec status: "FAILED" et claimedAt récent
NO_TRIP_PUBLISHED_7D	Reculer publishedAt de tous les trajets de plus de 7 jours
RETENTION_HELD_7D	Un Booking CANCELLED avec retentionDisposition: "HELD_FOR_MEDIATION" et closedAt il y a plus de 7 jours
REVERSAL_OPEN_48H	Un Booking avec payoutStatus: "REVERSED", payoutReversalResolution absent, updatedAt ancien

Déclencher.

```
// scripts/recette-alertes.ts - usage : ... [--email]
import redis from "../packages/libs/redis";
import { opsAlertsService } from "../apps/deal-service/src/routes/deal.routes";
(async () => {
  const { alerts } = await opsAlertsService.evaluate();
  console.log("règles actives :", alerts.map((a: { rule: string }) => a.rule));
  if (process.argv.includes("--email")) {
    console.log("emails envoyés pour :", await opsAlertsService.notifyNewAlerts(redis));
  }
  process.exit(0);
})();
```

Lire ou effacer le verrou de dédoublement :

```
// scripts/recette-alertes-verrou.ts
import redis from "../packages/libs/redis";
(async () => {
  const keys = await redis.keys("yamba:alerts:sent:*");
  console.log(keys.length ? keys : "aucun verrou");
  if (process.argv.includes("--reset") && keys.length) console.log("supprimés :", await
redis.del(...keys));
  process.exit(0);
})();
```

CRON-ALERTES-1 — Une alerte nouvelle déclenche un email au support

Objectif	Cas nominal
Prérequis	SUPPORT_EMAIL renseigné ; Mailpit vidé ; verrous Redis effacés

Étapes

1. Fabriquer la condition d'une règle — le plus simple : `OUTBOX_PARKED`, avec `seed-outbox.ts --with-poison` et le relais actif.
2. Lancer `recette-alertes.ts --email`.

Résultat attendu

- Le script liste `OUTBOX_PARKED` dans les règles actives **et** dans les règles notifiées.
- Un email arrive dans Mailpit, à l'adresse `SUPPORT_EMAIL`, en **français**, contenant le titre de l'alerte et un lien vers le back-office (`http://localhost:3001/pilotage`).
- Une clé `yamba:alerts:sent:OUTBOX_PARKED:<date du jour>` existe dans Redis, avec un TTL d'environ 48 h.
- La même alerte apparaît sur l'accueil du back-office et sur `/alerts`.

Verdict : ☐ conforme ☐ non conforme

CRON-ALERTE-2 — Rien sous le seuil, rien à envoyer

Objectif	Cas où la tâche ne doit pas agir
----------	---

Étapes

1. Remettre le jeu d'essai et les paramètres aux défauts (`seed-deals.ts`, `seed-settings.ts`).
2. S'assurer que le relais tourne et que l'outbox est vide de non publiés.
3. Vider Mailpit et les verrous.
4. Lancer `recette-alertes.ts --email`.

Résultat attendu

- Le script affiche une liste d'alertes vide (ou seulement des règles légitimement vraies sur ce jeu d'essai, à documenter).
- **Aucun** email dans Mailpit.
- Aucune clé `yamba:alerts:sent:*`.
- Le battement porte le résumé « aucune nouvelle alerte ».

Verdict : ☐ conforme ☐ non conforme

CRON-ALERTE-3 — Non-régression : une alerte, un email par jour

Objectif	Vérifier le dédoublement : la condition reste vraie pendant des jours, le support ne doit pas recevoir 24 emails
----------	--

Étapes

1. Jouer CRON-ALERTE-1 (un email reçu).
2. **Sans rien changer**, relancer `recette-alertes.ts --email` cinq fois.

Résultat attendu

- La liste des règles **actives** contient toujours `OUTBOX_PARKED` (la condition n'a pas disparu).
- La liste des règles **notifiées** est **vide** à chaque relance.
- Mailpit contient toujours **un seul** email.
- Le verrou Redis n'a pas été réécrit (le TTL décroît, il ne repart pas à 48 h — c'est le propre du `NX`).

Étape complémentaire : supprimer le verrou (`recette-alertes-verrou.ts --reset`) et relancer. Un second email doit alors partir — c'est ce qui se produira demain, à la bascule de date UTC.

Verdict : ☐ conforme ☐ non conforme

CRON-ALERTE-4 — Le seuil est bien lu dans les paramètres

Objectif	Vérifier qu'un seuil réglé dans le back-office change le comportement du cron
----------	---

Étapes

1. Fabriquer un versement en échec depuis **10 heures** — sous le seuil par défaut de 48 h.
2. Lancer `recette-alertes.ts` : `PAYOUT_FAILED_48H` ne doit **pas** apparaître.
3. Dans le back-office, `/settings`, ramener `alerts.payoutFailedHours` à **1** (motif d'au moins 20 caractères, obligatoire).
4. Attendre **plus de 30 secondes** (le lecteur de paramètres a un cache mémoire de 30 s dans chaque service), puis relancer.

Résultat attendu — `PAYOUT_FAILED_48H` apparaît maintenant. Un écart ici signifie soit que le seuil n'est pas branché, soit que le cache n'est pas expiré (relancer après 30 s avant de conclure).

Vérification complémentaire : le changement de paramètre a laissé **une ligne** `SETTING_CHANGED` dans le journal d'audit et un email aux `SUPER_ADMIN` (cahier 02).

Verdict : ☐ conforme ☐ non conforme

CRON-ALERTE-5 — Le verrou est posé même si l'email ne part pas

Objectif	Documenter un comportement subtil : le dédoublement Redis est posé avant le test de configuration de l'email
Gravité si écart	Mineure — comportement connu, à consigner s'il change

Étapes

1. Effacer les verrous, fabriquer une alerte.
2. Démarrer le deal-service **sans** configuration d'email (`EMAIL_PROVIDER=fake`, ou en vidant `SMTP_HOST` et `RESEND_API_KEY`), par le paquet construit.
3. Lancer le cron ou le script.

Résultat attendu — la règle est renvoyée comme « notifiée » et **le verrou est posé**, alors qu'aucun email n'est parti. Conséquence à connaître pour l'exploitation : une alerte survenue pendant une panne d'email **ne sera pas rejouée le jour même**. Si ce comportement devient inacceptable, c'est une évolution à porter au registre, pas une correction silencieuse.

Verdict : ☐ conforme ☐ non conforme

4.5 Notation : relances et révélation — `rating`

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/rating.cron.ts</code>
Horaires réels	<code>17 * * * *</code> — toutes les heures, à la dix-septième minute
Interrupteur	<code>RATING_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:rating</code>

Rubrique	Valeur
Garde de chevauchement	Drapeau <code>running</code>
Service appelé	<code>sendRatingReminders()</code> puis <code>revealElapsed()</code> dans <code>apps/deal-service/src/services/deal-rating.service.ts</code>

Ce qu'elle fait. Deux passes.

Passe 1 — les relances. Deux sous-passes, à **J+5** puis **J+7** après la complétion (`RATING_REMINDER_DAYS = [5, 7]`). Sélection : `status: "COMPLETED"`, `completedAt <= now - N jours`, `ratingWindowEndsAt > now` (la fenêtre est encore ouverte), et surtout **ratingRemindersSent exactement égal à l'indice** (0 pour la première relance, 1 pour la seconde). Un événement `booking.rating_reminder` est émis **par rôle qui n'a pas encore noté**, avec `reminderNumber` et `targetRole`. Le compteur `ratingRemindersSent` passe à `indice + 1`.

Passe 2 — la révélation. Sélection : `status: "COMPLETED"`, `ratingWindowEndsAt <= now`, `ratingsRevealedAt` absent ou nul. Le champ `ratingsRevealedAt` est posé, **et dans la même transaction** tous les `Review` de la réservation encore masqués reçoivent leur `revealedAt`. Un événement `booking.rating_revealed` (avec `revealedReason: "WINDOW_ELAPSED"`) n'est émis **que si au moins une des deux parties a noté** ; sinon la fenêtre se ferme en silence.

Attention aux noms de champs, ils se ressemblent et se confondent facilement : - sur `Booking` : `ratingsRevealedAt` (avec un s), `ratingRemindersSent` (entier), `ratingWindowEndsAt`, `shipperRatedAt`, `carrierRatedAt` ; - sur `Review` : `revealedAt`. Il n'existe **pas** de champ `ratingRemindedAt`.

Rendre une réservation éligible.

But	Champs à poser sur <code>Booking</code>
Première relance (J+5)	<code>status: "COMPLETED"</code> , <code>completedAt = now - 6 j</code> , <code>ratingWindowEndsAt = now + 8 j</code> , <code>ratingRemindersSent: 0</code>
Seconde relance (J+7)	idem, <code>completedAt = now - 8 j</code> , <code>ratingRemindersSent: 1</code>
Révélation	<code>status: "COMPLETED"</code> , <code>ratingWindowEndsAt = now - 1 h</code> , <code>ratingsRevealedAt</code> absent

```
// scripts/recette-notation-eligible.ts - usage : ... <bookingId> <r1|r2|reveal>
import prisma from "../packages/libs/prisma";
const [ID, MODE] = process.argv.slice(2);
const d = (n: number) => new Date(Date.now() + n * 86_400_000);
(async () => {
  const data =
    MODE === "r1"    ? { status: "COMPLETED" as const, completedAt: d(-6), ratingWindowEndsAt:
d(8), ratingRemindersSent: 0 }
    : MODE === "r2"    ? { status: "COMPLETED" as const, completedAt: d(-8), ratingWindowEndsAt:
d(6), ratingRemindersSent: 1 }
    :                   { status: "COMPLETED" as const, completedAt: d(-15), ratingWindowEndsAt:
d(-1), ratingsRevealedAt: null };
  const b = await prisma.booking.update({ where: { id: ID }, data });
  console.log(MODE, "->", b.completedAt, b.ratingWindowEndsAt, b.ratingRemindersSent,
b.ratingsRevealedAt);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-notation.ts
import { dealRatingService } from "../apps/deal-service/src/routes/deal.routes";
(async () => {
  console.log("relances :", await dealRatingService.sendRatingReminders());
  console.log("révélations :", await dealRatingService.revealElapsed());
  process.exit(0);
})();
```

CRON-NOTATION-1 — La première relance part à J+5

Étapes — `recette-notation-eligible.ts` <id> `r1`, vider Mailpit, lancer `recette-notation.ts`.

Résultat attendu

- Le compteur de relances renvoyé est ≥ 1 .
- `ratingRemindersSent` est passé de **0 à 1**.
- Dans l'outbox : **un** `booking.rating_reminder` par rôle n'ayant pas noté (donc deux si personne n'a noté), avec `reminderNumber: 1` et le bon `targetRole`.
- Après le relai : la notification in-app va au **rôle ciblé seulement** (règle `TARGET_ROLE` de la matrice), et l'email correspondant est dans Mailpit.

Verdict : ☐ conforme ☐ non conforme

CRON-NOTATION-2 — La révélation à la fin de la fenêtre

Étapes

- Faire noter **une seule** des deux parties (par l'écran, cahier 01, ou en posant `shipperRatedAt` et un `Review` avec `revealedAt: null`).
- `recette-notation-eligible.ts` <id> `reveal`.
- Lancer `recette-notation.ts`.

Résultat attendu

- `ratingsRevealedAt` est posé sur la réservation.
- Le `Review` existant a reçu son `revealedAt` — **dans la même transaction** : il ne doit pas exister d'état où la réservation est révélée mais l'avis encore masqué.
- Un événement `booking.rating_revealed` avec `revealedReason: "WINDOW_ELAPSED"`.
- La note apparaît sur le profil public du noté et `CarrierPage.ratingsAvg` / `ratingsCount` sont recalculés.

Verdict : ☐ conforme ☐ non conforme

CRON-NOTATION-3 — Ce que la tâche ne doit PAS faire

Trois cas à jouer, tous « ne doit pas agir » :

Cas	Préparation	Attendu
Fenêtre encore ouverte	<code>ratingWindowEndsAt = now + 3 j</code>	Aucune révélation ; <code>ratingsRevealedAt</code> reste absent
Relance déjà envoyée	<code>ratingRemindersSent: 1</code> alors qu'on est à J+5	Aucune relance : le filtre exige <code>ratingRemindersSent === indice</code>

Cas	Préparation	Attendu
Aucune note du tout	Fenêtre échue, ni <code>shipperRatedAt</code> ni <code>carrierRatedAt</code>	<code>ratingsRevealedAt</code> est posé (la fenêtre se ferme) mais aucun événement <code>booking.rating_revealed</code> n'est émis — donc aucun email, aucune notification. La fenêtre se ferme en silence, c'est voulu

Verdict : ☐ conforme ☐ non conforme

CRON-NOTATION-4 — Non-régression : jamais deux relances, jamais deux révélations

Étapes — après CRON-NOTATION-1 et CRON-NOTATION-2, relancer `recette-notation.ts` trois fois.

Résultat attendu

- Les compteurs renvoyés retombent à `0`.
- `ratingRemindersSent` n'a pas dépassé la valeur attendue.
- `ratingsRevealedAt` n'a pas été réécrit (même horodatage).
- Mailpit ne contient pas d'email supplémentaire.

La protection sur la révélation est un **verrou optimiste** : la mise à jour est conditionnée par `updatedAt` (la valeur lue). Deux instances qui liraient la même réservation ne pourraient pas la révéler deux fois — la seconde échouerait avec « Already revealed. » C'est le patron A85, et la raison pour laquelle il ne faut jamais modifier `updatedAt` à la main pendant ce scénario.

Verdict : ☐ conforme ☐ non conforme

CRON-NOTATION-5 — Le compteur avance même sans cible

Objectif	Documenter un comportement subtil : si les deux parties ont déjà noté, la passe de relance incrémente quand même <code>ratingRemindersSent</code> sans émettre d'événement
Gravité si écart	Mineure

Étapes — poser `shipperRatedAt` et `carrierRatedAt`, `completedAt = now - 6 j`, `ratingRemindersSent: 0`, puis lancer la passe.

Résultat attendu — `ratingRemindersSent` passe à 1, **zéro** événement `booking.rating_reminder`, zéro email. Conséquence : la seconde relance (J+7) ne partira pas non plus. C'est cohérent — plus personne n'a à noter — mais il faut le savoir en lisant un compteur.

Verdict : ☐ conforme ☐ non conforme

4.6 Récapitulatif quotidien — ops-digest

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/ops-digest.cron.ts</code>
Horaire réel	<code>0 8 * * *</code> — tous les jours à 08:00 UTC
Interrupteur	<code>OPS_DIGEST_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:ops-digest</code>
Garde de chevauchement	aucune — un seul envoi par jour, travail court

Rubrique	Valeur
Service appelé	collectOpsDigest() (deal-settlement) puis sendOpsDigest(digest, now) (apps/deal-service/src/services/ops-notify.service.ts)

Ce qu'elle fait. Elle constitue trois listes (100 lignes au maximum chacune) et envoie **un seul email** au support, en français :

Liste	Sélection
failed	status ∈ {COMPLETED, CANCELLED}, payoutStatus: "FAILED", updatedAt il y a plus de 24 h
reversed	payoutStatus: "REVERSED" et payoutReversalResolution absent ou nul (sans condition d'ancienneté)
held	status: "CANCELLED" et retentionDisposition: "HELD_FOR_MEDIATION" (sans condition d'ancienneté)

L'email n'est **pas** envoyé dans deux cas, dans cet ordre : l'email n'est pas configuré, ou **les trois listes sont vides**. Une seule ligne dans n'importe laquelle des trois suffit à envoyer.

Rendre le récapitulatif non vide. Modèle Booking :

```
// scripts/recette-digest-eligible.ts - usage : ... <bookingId> <failed|reversed|held>
import prisma from "../packages/libs/prisma";
const [ID, MODE] = process.argv.slice(2);
(async () => {
  const data =
    MODE === "failed" ? { status: "COMPLETED" as const, payoutStatus: "FAILED" as const,
      payoutFailureReason: "CARRIER_ACCOUNT_NOT_READY" }
    : MODE === "reversed" ? { payoutStatus: "REVERSED" as const, payoutFailureReason:
      "PROVIDER_REVERSED",
      payoutReversalResolution: null }
    : { status: "CANCELLED" as const, retentionDisposition:
      "HELD_FOR_MEDIATION",
      retentionCents: 1500, closedAt: new Date(Date.now() - 10 * 86_400_000)
    };
  await prisma.booking.update({ where: { id: ID }, data });
  // « failed » exige updatedAt de plus de 24 h : Prisma le remet à maintenant, il faut donc le
  forcer ensuite
  console.log("préparé :", MODE);
  process.exit(0);
})();
```

Piège spécifique au récapitulatif. La liste `failed` filtre sur **updatedAt de plus de 24 heures**, et `updatedAt` est un champ `@updatedAt` : toute écriture Prisma le remet à l'instant présent. Impossible donc de préparer ce cas avec un simple `update`. Deux solutions : (a) injecter une horloge avancée de 48 h à la lecture, (b) écrire `updatedAt` explicitement dans la même opération — Prisma le permet en le passant dans `data`, mais la valeur peut être écrasée selon le connecteur : vérifier après écriture. La méthode (a) est plus fiable.

Déclencher.

```
// scripts/recette-digest.ts
import { dealSettlementService } from "../apps/deal-service/src/routes/deal.routes";
import { sendOpsDigest } from "../apps/deal-service/src/services/ops-notify.service";
(async () => {
  const digest = await dealSettlementService.collectOpsDigest();
  console.log({ failed: digest.failed.length, reversed: digest.reversed.length, held:
digest.held.length });
  console.log("email envoyé :", await sendOpsDigest(digest, new Date()));
  process.exit(0);
})();
```

CRON-DIGEST-1 — Le récapitulatif part avec ses trois listes

Étapes — préparer au moins une réservation par catégorie, vider Mailpit, lancer `recette-digest.ts`.

Résultat attendu

- Le script affiche des longueurs non nulles et `email envoyé : true`.
- Un email dans Mailpit, adressé à `SUPPORT_EMAIL`, en français, daté du jour.
- Chaque ligne porte le corridor (`Ville → Ville`), le statut, l'identifiant de la réservation, un montant formaté en euros et un lien `http://localhost:3001/deals/<id>` — vers le **back-office**, jamais vers l'application membre.
- Montants : pour `failed` et `reversed`, `payoutAmountCents` à défaut `pricing.transportCents` ; pour `held`, `retentionCents`.

Verdict : ☐ conforme ☐ non conforme

CRON-DIGEST-2 — Rien à signaler, rien à envoyer

Étapes — repartir d'un jeu d'essai propre, vider Mailpit, lancer `recette-digest.ts`.

Résultat attendu — les trois listes sont vides, `email envoyé : false`, **aucun** email dans Mailpit. Un récapitulatif quotidien vide qui partirait quand même finirait par être ignoré par le support : c'est la raison de cette garde.

Verdict : ☐ conforme ☐ non conforme

CRON-DIGEST-3 — Non-régression : un seul email par passage

Étapes — après CRON-DIGEST-1, relancer `recette-digest.ts` deux fois.

Résultat attendu — deux emails supplémentaires arrivent. **C'est le comportement normal** : cette tâche n'a **aucun** dédoublement, contrairement aux alertes. Sa protection est son horaire (une fois par jour). Le point de recette est donc double :

1. En fonctionnement normal (`0 8 * * *`), le support reçoit **exactement un** récapitulatif par jour — à vérifier sur 48 h de fonctionnement réel, ou en lisant deux battements consécutifs distants de 24 h.
2. Si le cron était démarré deux fois (deux instances du `deal-service` sans coupure), le support recevrait **deux** récapitulatifs. À la différence du relais, il n'y a **pas de bail** ici. C'est une limite à connaître avant de passer à deux instances : consigner comme anomalie **majeure** si le déploiement cible est multi-instances.

Verdict : ☐ conforme ☐ non conforme

CRON-DIGEST-4 — L'erreur d'envoi remonte

Objectif	Documenter que <code>sendOpsDigest</code> n'avale pas ses erreurs, contrairement aux alertes
-----------------	--

Étapes — arrêter Mailpit (`docker stop yamba-mailpit`), préparer un récapitulatif non vide, lancer la tâche.

Résultat attendu — le cron consigne `Ops digest cron failed` et le **battement porte** `ok: false` avec le message d'erreur. C'est une bonne nouvelle : la panne est visible. Redémarrer Mailpit ensuite (`docker start yamba-mailpit`).

Verdict : ☐ conforme ☐ non conforme

4.7 Relance des messages non lus — `unread-reminder`

Rubrique	Valeur
Fichier	<code>apps/message-service/src/cron/unread-reminder.cron.ts</code>
Horaire réel	<code>* / 5 * * * *</code>
Interrupteur	<code>MESSAGING_REMINDER_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:message-service:unread-reminder</code>
Garde de chevauchement	aucune au niveau du cron — la protection est ailleurs, et elle est meilleure : un verrou optimiste par conversation et par rôle
Service appelé	<code>runOnce(now)</code> dans <code>apps/message-service/src/services/unread-reminder.service.ts</code> , règle pure dans <code>apps/message-service/src/lib/unread-reminder.rules.ts</code>

Ce qu'elle fait. Elle charge les conversations dont `lastMessageAt` est **plus vieux que le délai de relance** et **plus récent que 7 jours** (au-delà, relancer serait du bruit), au plus 200, puis, pour chacun des deux rôles, applique la règle pure `unreadReminderDue`. Cinq refus possibles, tous nommés :

Verdict	Signification
<code>NO_MESSAGE</code>	La conversation n'a pas de dernier message
<code>NOT_FROM_COUNTERPART</code>	Le dernier message vient du destinataire lui-même (ou du système) : on ne relance jamais l'auteur
<code>T00_RECENT</code>	Moins de <code>messaging.reminderDelayMinutes</code> (défaut 15 min)
<code>ALREADY_READ</code>	<code><rôle>LastReadAt</code> est postérieur ou égal au dernier message
<code>ALREADY_REMINDED</code>	La dernière relance est postérieure au dernier message : elle a déjà fait son travail
<code>RATE_LIMITED</code>	Moins de <code>messaging.reminderMinIntervalMinutes</code> (défaut 60 min) depuis la dernière relance

Si la relance est due, elle est d'abord **réclamée** par un `updateMany` conditionnel sur l'**ancienne valeur** de `shipperRemindedAt` ou `carrierRemindedAt` : deux instances ne peuvent pas relancer deux fois, sans le moindre Redis. Ensuite seulement l'email part — et il ne part pas si le destinataire est effacé, si son adresse est en liste de suppression, ou s'il a coupé `messagingReminderEmails`.

L'email ne contient jamais le texte du message : prénom, prénom de l'autre partie, corridor, et un lien vers le fil.

Rendre une conversation éligible. Modèle `Conversation`, champs `lastMessageAt`, `lastMessageAuthorRole`, `shipperLastReadAt` / `carrierLastReadAt`, `shipperRemindedAt` / `carrierRemindedAt`.


```
// scripts/recette-reliance-eligible.ts - usage : ... <conversationId> <SHIPPER|CARRIER>
import prisma from "../packages/libs/prisma";
const [ID, ROLE] = process.argv.slice(2); // ROLE = le DESTINATAIRE de la reliance
(async () => {
  const auteur = ROLE === "SHIPPER" ? "CARRIER" : "SHIPPER";
  const c = await prisma.conversation.update({
    where: { id: ID },
    data: {
      lastMessageAt: new Date(Date.now() - 30 * 60_000), // 30 min > délai de 15 min
      lastMessageAuthorRole: auteur,
      ...(ROLE === "SHIPPER"
        ? { shipperLastReadAt: new Date(Date.now() - 3 * 3_600_000), shipperRemindedAt: null }
        : { carrierLastReadAt: new Date(Date.now() - 3 * 3_600_000), carrierRemindedAt: null }),
    },
  });
  console.log("conversation prête :", c.id, c.lastMessageAt, c.lastMessageAuthorRole);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-reliance.ts
import { makeUnreadReminderService } from "../apps/message-service/src/services/unread-reminder.service";
(async () => { console.log(await makeUnreadReminderService().runOnce()); process.exit(0); })();
```

CRON-RELANCE-1 — Le message non lu depuis 15 minutes déclenche une reliance

Étapes — vider Mailpit, préparer la conversation avec le script, lancer `recette-reliance.ts`.

Résultat attendu

- Le script renvoie `{ scanned: n, sent: 1, failed: 0 }`.
- En base, `<rôle>RemindedAt` est renseigné à l'instant du passage.
- Un email dans Mailpit, adressé au **destinataire** (jamais à l'auteur), dans **sa** langue (`User.preferredLocale`), contenant le prénom de l'autre partie, le corridor `Ville → Ville`, et un lien `.../dashboard/messages?conversation=<id>`.
- **L'email ne contient pas le texte du message.** C'est une exigence, pas un détail : le fil reste dans l'application.
- Le battement porte « 1 reliance(s), 0 échec(s) ».

Verdict : ☐ conforme ☐ non conforme

CRON-RELANCE-2 — Les six cas où la reliance ne doit PAS partir

À jouer un par un, en repartant à chaque fois d'une conversation propre. Aucun email ne doit arriver dans Mailpit.

Cas	Préparation	Verdict attendu de la règle
Message trop récent	<code>lastMessageAt = now - 5 min</code>	<code>TOO_RECENT</code>
Message de soi-même	<code>lastMessageAuthorRole</code> = le rôle du destinataire	<code>NOT_FROM_COUNTERPART</code>
Message système	<code>lastMessageAuthorRole</code> = "SYSTEM"	<code>NOT_FROM_COUNTERPART</code>
Déjà lu	<code><rôle>LastReadAt >= lastMessageAt</code>	<code>ALREADY_READ</code>

Cas	Préparation	Verdict attendu de la règle
Déjà relancé pour ce message	<code><rôle>RemindedAt >= lastMessageAt</code>	ALREADY_REMINDED
Trop tôt après la relance précédente	<code><rôle>RemindedAt = now - 10 min</code> , un message plus récent	RATE_LIMITED

Cas complémentaire à jouer aussi : **conversation silencieuse depuis plus de 7 jours** (`lastMessageAt = now - 10 j`). Elle n'est même pas chargée par la requête : aucun email, et le compteur `scanned` ne l'inclut pas.

Verdict : ☐ conforme ☐ non conforme

CRON-RELANCE-3 — Non-régression : le verrou optimiste empêche le double envoi

Objectif	Prouver que deux passages simultanés n'envoient qu'un email
----------	---

Étapes

1. Préparer une conversation éligible.
2. Lancer **deux** exécutions du script **en parallèle** : `sh npx tsx --env-file=.env scripts/recette-relance.ts & \ npx tsx --env-file=.env scripts/recette-relance.ts & wait`

Résultat attendu

- L'une des deux renvoie `sent: 1`, l'autre `sent: 0`.
- **Un seul** email dans Mailpit.
- `<rôle>RemindedAt` porte un seul horodatage.

Le mécanisme : la réclamation est un `updateMany` conditionné par l'**ancienne** valeur du champ (`{ champ: valeurLue }`, ou OR: `[{ champ: null }, { champ: { isSet: false } }]` si le champ était absent). Le premier qui écrit gagne, le second obtient `count: 0` et passe son chemin. C'est ce qui rend cette tâche sûre à deux instances **sans Redis**.

Verdict : ☐ conforme ☐ non conforme

CRON-RELANCE-4 — Les délais sont lus dans les paramètres

Étapes

1. Dans le back-office `/settings`, porter `messaging.reminderDelayMinutes` à **1** et `messaging.reminderMinIntervalMinutes` à **5** (motif ≥ 20 caractères).
2. Attendre 30 secondes (cache du lecteur de paramètres).
3. Préparer une conversation avec `lastMessageAt = now - 2 min`.
4. Lancer la relance.

Résultat attendu — la relance part, alors qu'elle ne serait pas partie avec le délai par défaut de 15 minutes.

Vérification complémentaire — tenter de régler `messaging.reminderMinIntervalMinutes` **en dessous** de `messaging.reminderDelayMinutes` : le formulaire doit **refuser** (« L'intervalle entre deux relances doit être supérieur ou égal au délai de relance. »). Une garde de cohérence entre deux paramètres, exactement comme un invariant métier.

Verdict : ☐ conforme ☐ non conforme

CRON-RELANCE-5 — Le destinataire qui a coupé les relances

Objectif	Vérifier la préférence membre <code>messagingReminderEmails</code>
----------	--

Étapes — poser `messagingReminderEmails: false` sur le destinataire, préparer une conversation éligible, lancer la relance.

Résultat attendu

- **Aucun** email.
- Mais `<rôle>RemindedAt` **est quand même posé** : le verrou est réclamé avant l'envoi. Conséquence à connaître : la conversation ne sera pas réexaminée en boucle. C'est voulu.

Verdict : ☐ conforme ☐ non conforme

4.8 Purge des conversations — `conversation-retention`

Rubrique	Valeur
Fichier	<code>apps/message-service/src/cron/conversation-retention.cron.ts</code>
Horaire réel	<code>30 3 * * *</code> — tous les jours à 03:30
Interrupteur	<code>MESSAGING_RETENTION_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:message-service:conversation-retention</code>
Garde de chevauchement	aucune ; la suppression est idempotente par nature (une conversation supprimée n'est plus candidate)
Service appelé	<code>purgeOnce(now)</code> dans <code>apps/message-service/src/services/conversation-retention.service.ts</code> , règle pure dans <code>apps/message-service/src/lib/conversation-retention.rules.ts</code>
Paramètre	<code>messaging.retentionDays</code> — défaut 365 , bornes 30 à 1095

Ce qu'elle fait. Elle charge au plus 100 conversations dont `updatedAt` est antérieur à `now - retentionDays`, va chercher leur réservation, et applique la règle pure. Une conversation est purgeable si **deux conditions** sont réunies :

1. la réservation est dans un **statut terminal** (`COMPLETED`, `DECLINED`, `EXPIRED`, `CANCELLED`) ;
2. la plus **tardive** des deux dates — fin du deal (`completedAt` à défaut `closedAt`) et dernière activité du fil (`updatedAt`) — est antérieure à `now - retentionDays`.

Si la réservation est introuvable (supprimée), la conversation est traitée comme un deal terminé : elle n'a plus de raison d'être.

La suppression se fait en **une transaction** de quatre opérations, dans cet ordre : `PhoneReveal`, `Meetup`, `Message`, puis la `Conversation` elle-même. Les **Report** ne sont **pas** touchés : ce sont des dossiers de modération, pas des propos.

Rendre une conversation purgeable. Modèles `Conversation` (`updatedAt`) et `Booking` (`status`, `completedAt` / `closedAt`).

```
// scripts/recette-purgefil-eligible.ts - usage : ... <conversationId>
import prisma from "../packages/libs/prisma";
const ID = process.argv[2];
const vieux = new Date(Date.now() - 400 * 86_400_000); // > 365 jours
(async () => {
  const c = await prisma.conversation.findUnique({ where: { id: ID } });
  if (!c) throw new Error("conversation introuvable");
  await prisma.booking.update({ where: { id: c.bookingId }, data: { status: "COMPLETED",
completedAt: vieux } });
  // updatedAt est @updatedAt : l'écriture ci-dessus le remettrait à maintenant côté conversation.
  // On force donc la conversation en dernier, via une écriture explicite du champ.
  await prisma.conversation.update({ where: { id: ID }, data: { updatedAt: vieux } as never });
  const apres = await prisma.conversation.findUnique({ where: { id: ID } });
  console.log("conversation prête :", apres?.id, apres?.updatedAt);
  process.exit(0);
})();
```

Si l'écriture explicite de `updatedAt` est ignorée par le connecteur, utiliser à la place l'**horloge injectée** :
`makeConversationRetentionService(() => new Date(Date.now() + 400 * 86_400_000)).purgeOnce()`. C'est plus fiable et cela n'altère aucune donnée.

Déclencher.

```
// scripts/recette-purgefil.ts - argument facultatif : un décalage en jours
import { makeConversationRetentionService } from "../apps/message-service/src/services/conversation-
retention.service";
const j = Number(process.argv[2] ?? 0);
(async () => {
  const clock = () => new Date(Date.now() + j * 86_400_000);
  console.log(await makeConversationRetentionService(clock).purgeOnce());
  process.exit(0);
})();
```

CRON-PURGEFIL-1 — La conversation d'un deal terminé depuis plus d'un an disparaît

Étapes

1. Relever, pour la conversation ciblée : le nombre de `Message`, de `Meetup`, de `PhoneReveal`, et l'existence d'un `Report` visant l'un de ses messages (en créer un si besoin, par l'écran de signalement, cahier 01).
2. Rendre la conversation purgeable.
3. Lancer `recette-purgefil.ts`.

Résultat attendu

- Le script renvoie `{ examined: n, purged: ≥ 1 }`.
- La `Conversation` n'existe plus.
- **Zéro** `Message`, `Meetup`, `PhoneReveal` restant pour cet identifiant de conversation.
- **Le Report existe toujours**, avec son statut et son motif. Il a perdu son corps de message (le message a disparu) mais le dossier de modération survit.
- Le battement porte « 1 conversation(s) purgée(s) sur n ».

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEFIL-2 — Les trois cas où la conversation ne doit PAS être purgée

Cas	Préparation	Attendu
Deal encore vivant	<code>Booking.status = "ACCEPTED"</code> (ou <code>PICKED_UP</code> , <code>DELIVERED</code>), conversation très ancienne	Rien n'est purgé : <code>BOOKING_TERMINAL_STATUSES</code> ne contient pas ces statuts
Deal en litige	<code>Booking.status = "DISPUTED"</code> , conversation très ancienne	Rien n'est purgé — la médiation a besoin du fil, quel que soit son âge
Activité récente	Deal <code>COMPLETED</code> il y a deux ans, mais <code>Conversation.updatedAt</code> d'hier	Rien n'est purgé : l'ancre est la plus tardive des deux dates

Ce troisième cas est le plus facile à casser par une régression : si quelqu'un remplaçait l'ancre par la seule date de fin du deal, une conversation encore active serait effacée sous les yeux de ses deux membres.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEFIL-3 — Non-régression : rejouer ne casse rien

Étapes — après CRON-PURGEFIL-1, relancer `recette-purgefil.ts` deux fois.

Résultat attendu — `{ examined: 0, purged: 0 }` (ou des conversations non concernées), aucune erreur, aucun effet de bord. La transaction de suppression est atomique : il ne peut pas exister d'état intermédiaire où les messages sont partis mais la conversation reste.

Vérification complémentaire : provoquer un échec au milieu (par exemple en coupant la base pendant l'exécution sur un gros lot) et vérifier qu'aucune conversation n'est à moitié purgée.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEFIL-4 — La durée est lue dans les paramètres

Étapes — porter `messaging.retentionDays` à **30** dans le back-office, attendre 30 secondes, préparer une conversation terminée il y a 45 jours, lancer la purge.

Résultat attendu — la conversation est purgée, alors qu'elle ne l'aurait pas été avec les 365 jours par défaut. Puis remettre le paramètre à 365 (`seed-settings.ts`).

Verdict : ☐ conforme ☐ non conforme

4.9 Purge des événements publiés — `outbox-retention` (deal et message)

Rubrique	Valeur
Fichiers	<code>apps/deal-service/src/cron/outbox-retention.cron.ts</code> et <code>apps/message-service/src/cron/outbox-retention.cron.ts</code> (code identique)
Horaire réel	<code>55 3 * * *</code> — tous les jours à 03:55, pour les deux
Interrupteur	<code>OUTBOX_RETENTION_CRON_ENABLED=false</code> — la même variable coupe les deux
Noms des battements	<code>yamba:cron:deal-service:outbox-retention</code> et <code>yamba:cron:message-service:outbox-retention</code>
Garde de chevauchement	aucune ; un seul <code>deleteMany</code> borné par date
Fonction	<code>purgePublishedOutbox(aggregateType, now)</code>

Rubrique	Valeur
Paramètre	retention.outboxPublishedDays — défaut 90 , bornes 7 à 365

Ce qu'elle fait. Un seul `deleteMany` :

```
where: { aggregateType: <"booking" | "conversation">, publishedAt: { lt: now - N jours } }
```

Deux propriétés en découlent, et ce sont **les deux points de recette essentiels** :

1. **Chaque service ne purge que son propre type d'agrégat.** Le deal-service passe `"booking"`, le message-service passe `"conversation"` (câblé dans les `main.ts` respectifs). Aucun des deux ne peut effacer les événements de l'autre.
2. **Un événement jamais publié n'est jamais supprimé.** Le filtre porte sur `publishedAt: { lt: ... }` ; un événement dont `publishedAt` est `null` **ou absent** n'entre dans aucune borne de date. Un événement **parqué** (dix tentatives échouées, jamais publié) reste donc en base **indéfiniment** : c'est la piste d'audit, c'est intentionnel, et c'est une exigence de conformité (§ 8).

La règle pure correspondante, `isOutboxEventPurgeable` dans `packages/libs/retention/index.ts`, dit la même chose en une ligne : `!!e.publishedAt && olderThan(e.publishedAt, now, days)`.

Rendre des événements purgeables. Modèle `OutboxEvent`, champ `publishedAt`.

```
// scripts/recette-purgeout-eligible.ts
import prisma from "../packages/libs/prisma";
const vieux = new Date(Date.now() - 200 * 86_400_000); // > 90 jours
(async () => {
  // 1. Trois événements booking publiés, très anciens → doivent être purgés
  const publies = await prisma.outboxEvent.findMany({ where: { aggregateType: "booking", NOT: {
publishedAt: null } }, take: 3, select: { id: true } });
  for (const e of publies) await prisma.outboxEvent.update({ where: { id: e.id }, data: {
publishedAt: vieux } });
  // 2. Un événement parqué, très ancien, JAMAIS publié → ne doit JAMAIS être purgé
  const parque = await prisma.outboxEvent.create({
    data: { aggregateType: "booking", aggregateId: publies[0]?.id ?? "000000000000000000000000",
      eventType: "booking.requested", payload: { poison: true },
      correlationId: "recette-parque", occurredAt: vieux, publishedAt: null,
      attempts: 10, lastError: "recette : événement parqué", lastErrorAt: vieux },
  });
  console.log("publiés vieillis :", publies.length, "| parqué créé :", parque.id);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-purgeout.ts - usage : ... <booking|conversation>
import { purgePublishedOutbox } from "../apps/deal-service/src/cron/outbox-retention.cron";
(async () => {
  const type = process.argv[2] ?? "booking";
  console.log(type, "→", await purgePublishedOutbox(type), "événement(s) purgé(s)");
  process.exit(0);
})();
```

CRON-PURGEOUT-1 — Les événements publiés et anciens partent

Étapes — préparer avec le script ci-dessus, relever le total de `OutboxEvent`, lancer `recette-purgeout.ts booking`.

Résultat attendu

- Le script renvoie **3** (les trois événements vieilliss).
- Le total de la collection a diminué de 3.
- Le battement `deal-service:outbox-retention` porte « 3 événement(s) publié(s) purgé(s) ».
- Le bloc **Outbox** de la page « État des services » ne change pas ses compteurs `unpublished` et `parked` : la purge ne touche que du publié.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEOUT-2 — L'événement parké survit à la purge

Objectif	La garantie d'audit la plus importante de cette tâche
Gravité si écart	Bloquante

Étapes

1. S'assurer que l'événement parké créé plus haut existe (`correlationId: "recette-parque"` , `publishedAt: null` , `attempts: 10` , `occurredAt` il y a 200 jours).
2. Lancer `recette-purgeout.ts booking` **trois fois**.
3. Le rechercher en base.

Résultat attendu — l'événement **existe toujours**, avec ses dix tentatives, son `lastError` et sa charge utile intacte. Il est plus vieux que n'importe quelle durée de conservation, il n'est jamais purgé. C'est ce qui permet, six mois après, de comprendre pourquoi un membre n'a jamais reçu son email.

Variante à jouer aussi : un événement `publishedAt: null` **absent du document** (jamais écrit) plutôt que posé à `null`. Il doit survivre aussi — c'est le piège Mongo, et le filtre le gère parce qu'il ne mentionne que `lt`.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEOUT-3 — Chaque service ne purge que son domaine

Objectif	Vérifier le cloisonnement par <code>aggregateType</code>
-----------------	--

Étapes

1. Vieillir **des deux côtés** : trois événements `booking` publiés et trois événements `conversation` publiés, tous avec `publishedAt` il y a 200 jours.
2. Lancer `recette-purgeout.ts booking` **seulement**.
3. Compter par type d'agrégat (`recette-outbox.ts`).

Résultat attendu

- Les trois `booking` ont disparu.
- Les trois `conversation` sont **intacts**.
- Puis lancer `recette-purgeout.ts conversation` : les trois `conversation` partent à leur tour.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEOUT-4 — Le récent ne part pas, et la durée est paramétrable

Étapes

1. Vérifier qu'un événement publié **hier** n'est pas purgé (durée par défaut 90 jours).
2. Porter `retention.outboxPublishedDays` à **7** dans le back-office, attendre 30 secondes.
3. Vieillir un événement publié à 10 jours, relancer la purge.

Résultat attendu — il est purgé avec 7 jours, il ne l'était pas avec 90. Remettre ensuite le paramètre au défaut.

Verdict : ☐ conforme ☐ non conforme

CRON-PURGEOUT-5 — Non-régression : rejouer ne supprime rien de plus

Étapes — relancer la purge cinq fois de suite après CRON-PURGEOUT-1.

Résultat attendu — 0 à chaque fois, aucun message d'erreur. Un `deleteMany` borné par date est idempotent par construction.

Verdict : ☐ conforme ☐ non conforme

4.10 Effacement du destinataire — `recipient-redaction`

C'est une tâche de conformité (RGPD, D63 5A, RGP-02). Le destinataire d'un colis est un **tiers sans compte** : il n'a jamais rien signé, ses coordonnées ne doivent pas rester indéfiniment dans la base.

Rubrique	Valeur
Fichier	<code>apps/deal-service/src/cron/recipient-redaction.cron.ts</code>
Horaire réel	<code>40 3 * * *</code> — tous les jours à 03:40
Interrupteur	<code>RECIPIENT_REDACTION_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:deal-service:recipient-redaction</code>
Garde de chevauchement	aucune ; l'idempotence vient du marqueur <code>recipientRedactedAt</code>
Service appelé	<code>runOnce(now)</code> dans <code>apps/deal-service/src/services/recipient-redaction.service.ts</code> , règle pure dans <code>apps/deal-service/src/lib/recipient-redaction.rules.ts</code>
Paramètre	<code>privacy.recipientRetentionDays</code> — défaut 30 , bornes 7 à 365

Ce qu'elle fait. Elle charge au plus 200 réservations dont le statut est **terminal** (`COMPLETED`, `DECLINED`, `EXPIRED`, `CANCELLED`), dont `recipientRedactedAt` est absent ou nul, et dont `completedAt` ou `closedAt` est antérieur à `now - N jours`. Elle repasse ensuite chaque candidate à la règle pure (le OR de Prisma laisse passer des cas où seule une des deux dates est ancienne), puis **remplace en bloc** l'objet `recipient` :

Champ	Valeur après effacement
<code>recipient.firstName</code>	<code>"_"</code> (tiret cadratin U+2014, pas un tiret ASCII)
<code>recipient.lastName</code>	<code>"_"</code>
<code>recipient.phoneE164</code>	<code>"000000000000"</code>
<code>recipient.email</code>	<code>null</code>
<code>recipientRedactedAt</code>	l'instant du passage

Les valeurs ne sont **jamais** vidées ou mises à `null` sur les champs obligatoires : le type Prisma exige des chaînes. C'est le même principe que pour l'effacement d'un compte membre (`erased+<id>@anonymised.invalid`).

Aucun événement d'outbox, aucune transaction, aucun kilo : c'est une écriture par réservation.

Rendre une réservation éligible. Modèle `Booking`, champs `status`, `completedAt` ou `closedAt`, et `recipientRedactedAt` (qui doit être absent).

```
// scripts/recette-destinataire-eligible.ts - usage : ... <bookingId>
import prisma from "../packages/libs/prisma";
const ID = process.argv[2];
(async () => {
  const b = await prisma.booking.update({
    where: { id: ID },
    data: { status: "COMPLETED", completedAt: new Date(Date.now() - 60 * 86_400_000),
recipientRedactedAt: null },
  });
  console.log("avant effacement :", b.recipient, b.completedAt, b.recipientRedactedAt);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-destinataire.ts
import { makeRecipientRedactionService } from "../apps/deal-service/src/services/recipient-
redaction.service";
(async () => { console.log(await makeRecipientRedactionService().runOnce()); process.exit(0); })();
```

CRON-DESTINATAIRE-1 — Le tiers est effacé après le délai

Étapes

1. Relever les quatre champs de `recipient` avant : ils doivent contenir un vrai prénom, un vrai nom, un vrai numéro.
2. `recette-destinataire-eligible.ts <id>`.
3. Lancer `recette-destinataire.ts`.

Résultat attendu

- Le script renvoie `{ examined: n, redacted: ≥ 1 }`.
- Les quatre champs valent exactement `"_"`, `"_"`, `"_+000000000000"`, `null`.
- `recipientRedactedAt` est renseigné.
- **Le reste de la réservation est intact** : instantané de prix, colis, trajet, jalons, code de livraison, litiges, avis. On efface un tiers, pas un dossier.
- La page de suivi publique du colis (D69) ne montre plus le prénom du destinataire.
- Le battement porte « 1 destinataire(s) effacé(s) sur n ».

Verdict : ☐ conforme ☐ non conforme

CRON-DESTINATAIRE-2 — Les trois cas où le tiers ne doit PAS être effacé

Cas	Préparation	Attendu
Deal encore vivant	<code>status: "PICKED_UP"</code> , <code>deliveredAt</code> ancien	Rien : le statut n'est pas terminal. Un colis en cours a besoin de son destinataire

Cas	Préparation	Attendu
Deal terminé mais récent	<code>status: "COMPLETED", completedAt = now - 10 j</code> (délai 30 j)	Rien : le délai n'est pas écoulé. Un litige ou une preuve de remise peut encore en avoir besoin
Déjà effacé	<code>recipientRedactedAt</code> déjà posé	Rien : la réservation n'est même pas candidate

Verdict : ☐ conforme ☐ non conforme

CRON-DESTINATAIRE-3 — Non-régression : jamais deux effacements

Étapes — après CRON-DESTINATAIRE-1, relancer trois fois.

Résultat attendu

- `redacted`: 0 à chaque fois.
- `recipientRedactedAt` **inchangé** (même horodatage qu'au premier passage).
- Le contenu de `recipient` inchangé.

Le marqueur `recipientRedactedAt` est à la fois la preuve de conformité (« effacé le ... ») et la garde d'idempotence. Attention au piège Mongo : le filtre de sélection utilise bien `OR: [{ recipientRedactedAt: null }, { ... isSet: false }]`. Un test qui poserait seulement `null` ne prouverait rien sur les documents anciens où le champ n'existe pas.

Verdict : ☐ conforme ☐ non conforme

CRON-DESTINATAIRE-4 — Le délai est lu dans les paramètres, et il est borné

Étapes

1. Porter `privacy.recipientRetentionDays` à **7** (le minimum autorisé), attendre 30 secondes.
2. Préparer une réservation terminée il y a 10 jours.
3. Lancer la tâche.

Résultat attendu — le tiers est effacé, alors qu'il ne l'aurait pas été avec 30 jours.

Vérification complémentaire — tenter de régler le paramètre à **3** dans le back-office : le formulaire doit refuser (borne minimale de 7). Et à **400** : refus aussi (borne maximale de 365). Les bornes du catalogue sont une protection contre une conservation abusive comme contre un effacement prématuré.

Verdict : ☐ conforme ☐ non conforme

4.11 Conservation générale — `retention` (notification-service)

Rubrique	Valeur
Fichier	<code>apps/notification-service/src/cron/retention.cron.ts</code>
Horaire réel	<code>50 3 * * *</code> — tous les jours à 03:50
Interrupteur	<code>RETENTION_CRON_ENABLED=false</code>
Nom du battement	<code>yamba:cron:notification-service:retention</code>
Garde de chevauchement	aucune ; trois <code>deleteMany</code> bornés par date, lancés en parallèle
Service appelé	<code>makeRetentionService().runOnce(now)</code> dans le même fichier

Ce qu'elle fait. Trois suppressions, chacune avec sa propre durée lue dans les paramètres :

Collection	Champ de date	Paramètre	Défaut
Notification	createdAt	retention.notificationsDays	365 j
EmailDelivery	claimedAt	retention.emailDeliveriesDays	365 j
ConsumedEvent	claimedAt	retention.consumedEventsDays	90 j

Les notifications sont supprimées **qu'elles soient lues ou non** : la durée est le seul critère. Les traces d'email ne contiennent **jamais** le contenu de l'email — seulement le gabarit, le statut, l'erreur éventuelle, le fournisseur et l'identifiant du message.

Le troisième cas mérite une attention particulière. `ConsumedEvent` est le **registre d'idempotence des consommateurs** : c'est lui qui empêche qu'un même événement rejoué produise deux notifications. Le purger, c'est accepter qu'un événement plus vieux que 90 jours, s'il était rejoué, soit **retraité**. C'est un arbitrage assumé (la rétention du courtier est de 7 jours, un rejeu à 90 jours n'a pas de chemin naturel), mais il faut le savoir avant de baisser ce paramètre.

Rendre des lignes purgeables.

```
// scripts/recette-conservation-eligible.ts
import prisma from "../packages/libs/prisma";
const vieux = (j: number) => new Date(Date.now() - j * 86_400_000);
(async () => {
  const n = await prisma.notification.findMany({ take: 3, select: { id: true } });
  for (const x of n) await prisma.notification.update({ where: { id: x.id }, data: { createdAt: vieux(400) } });
  const e = await prisma.emailDelivery.findMany({ take: 2, select: { id: true } });
  for (const x of e) await prisma.emailDelivery.update({ where: { id: x.id }, data: { claimedAt: vieux(400) } });
  const c = await prisma.consumedEvent.findMany({ take: 2, select: { id: true } });
  for (const x of c) await prisma.consumedEvent.update({ where: { id: x.id }, data: { claimedAt: vieux(120) } });
  console.log("vieillis :", n.length, "notifications,", e.length, "traces d'email,", c.length, "événements consommés");
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-conservation.ts
import { makeRetentionService } from "../apps/notification-service/src/cron/retention.cron";
(async () => { console.log(await makeRetentionService().runOnce()); process.exit(0); })();
```

CRON-CONSERV-1 — Les trois collections sont purgées

Étapes — compter les trois collections, préparer, lancer, recompter.

Résultat attendu

- Le script renvoie `{ notifications: 3, emailDeliveries: 2, consumedEvents: 2 }`.
- Les trois compteurs en base ont baissé d'autant.
- Le battement porte « 3 notification(s), 2 trace(s) d'email, 2 événement(s) consommé(s) ».

Verdict : ☐ conforme ☐ non conforme

CRON-CONSERV-2 — Le récent est conservé, et chaque durée est indépendante

Étapes

1. Vieillir une notification à **100 jours** et un `ConsumedEvent` à **100 jours**.
2. Lancer la tâche avec les durées par défaut (365 / 365 / 90).

Résultat attendu

- La notification (100 j < 365 j) est **conservée**.
- Le `ConsumedEvent` (100 j > 90 j) est **supprimé**.

C'est la preuve que les trois durées sont bien lues séparément et appliquées à la bonne collection — une inversion de paramètres serait invisible autrement.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSERV-3 — Non-régression : purger n'efface pas ce qui n'est pas à elle

Objectif	Vérifier que cette tâche ne touche ni l'outbox, ni les conversations, ni les réservations
-----------------	---

Étapes — relever les totaux de `OutboxEvent`, `Conversation`, `Message`, `Booking` avant et après un passage sur un jeu volontairement vieilli.

Résultat attendu — ces quatre totaux sont **strictement identiques**. Chaque service purge ses propres collections, jamais celles d'un autre. Une purge qui déborde de son domaine est une anomalie **bloquante**.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSERV-4 — Le registre d'idempotence purgé rouvre la porte au retraitement

Objectif	Documenter la conséquence réelle de <code>retention.consumedEventsDays</code>
Gravité si écart	Mineure (comportement attendu), mais l'expérience doit être faite au moins une fois

Étapes

1. Choisir un événement déjà consommé : relever son `eventId` et le `ConsumedEvent` correspondant (statut `PROCESSED`).
2. Supprimer ce `ConsumedEvent` (c'est ce que ferait la purge à 90 jours).
3. Rejouer l'événement (voir CRON-CONSO-2 pour la méthode : remettre `publishedAt` à `null` sur la ligne d'outbox et laisser le relais republier).

Résultat attendu — l'événement est **retraité**. La notification, elle, n'est **pas** dupliquée : la matérialisation passe par un `upsert` sur la clé unique `[eventId, userId]`. En revanche, l'email **peut** repartir si sa propre trace `EmailDelivery` a aussi été purgée. C'est la conséquence à connaître : les deux durées (`consumedEventsDays` 90 j et `emailDeliveriesDays` 365 j) ne sont pas alignées, ce qui protège en pratique.

Verdict : ☐ conforme ☐ non conforme

4.12 Rappels d'inscription Voyageur — onboarding-reminder

Rubrique	Valeur
Fichier	<code>apps/auth-service/src/cron/onboarding-reminder.cron.ts</code>
Horaire réel	<code>0 * * * *</code> — toutes les heures, à la minute zéro
Interrupteur	<code>ONBOARDING_REMINDER_CRON_ENABLED=false</code> (A148 — contredit le chapitre 9 de la documentation technique, voir DIV-1)
Nom du battement	<code>yamba:cron:auth-service:onboarding-reminder</code>
Garde de chevauchement	Drapeau <code>running</code>
Service appelé	<code>processOnboardingReminders(now)</code> , règle pure dans <code>apps/auth-service/src/utils/onboarding-reminder.rules.ts</code>

Ce qu'elle fait. Elle charge au plus 200 utilisateurs en `carrierStatus: "ONBOARDING"`, non supprimés, dont l'adresse n'est pas en liste de suppression, et dont la `CarrierPage` a moins de 3 rappels envoyés. Pour chacun, la règle pure décide du rappel à envoyer :

Rappel	Déclenchement	Objet
1	24 h après la création de la <code>CarrierPage</code>	« Plus qu'une étape pour devenir Voyageur ! »
2	72 h	« Ton profil Voyageur t'attend... »
3	168 h (7 jours)	« Dernière chance de finaliser ton profil »

Quatre refus supplémentaires, tous dans la règle pure :

- compte supprimé ou adresse en liste de suppression (règle générale D35) ;
- statut différent de `ONBOARDING` (l'inscription s'est terminée entre-temps) ;
- `CarrierPage` **plus vieille que 30 jours** (`ONBOARDING_REMINDER_MAX_AGE_DAYS`) — on ne réveille pas un compte abandonné avec une « dernière chance » incongrue ;
- **moins de 12 heures** depuis le rappel précédent (`ONBOARDING_REMINDER_MIN_INTERVAL_HOURS`).

Après un envoi, `CarrierPage.lastReminderSentAt` et `CarrierPage.reminderCount` sont mis à jour.

Ce garde-fou des 30 jours est né du branchement lui-même : allumer un cron dormant depuis des mois, sans borne d'âge, aurait envoyé d'un coup une vague de « dernière chance » à des comptes abandonnés depuis longtemps. C'est un point de recette à part entière (CRON-ONBOARD-3).

Rendre un compte éligible. Modèles `User` (`carrierStatus`, `isDeleted`, `emailSuppressedAt`) et `CarrierPage` (`createdAt`, `reminderCount`, `lastReminderSentAt`).

```
// scripts/recette-onboard-eligible.ts - usage : ... <email> <1|2|3>
import prisma from "../packages/libs/prisma";
const [EMAIL, ETAPE] = process.argv.slice(2);
const h = (n: number) => new Date(Date.now() - n * 3_600_000);
const age = { "1": 30, "2": 80, "3": 180 }[ETAPE] ?? 30;
(async () => {
  const u = await prisma.user.update({
    where: { email: EMAIL },
    data: { carrierStatus: "ONBOARDING", isDeleted: false, emailSuppressedAt: null },
  });
  const p = await prisma.carrierPage.update({
    where: { userId: u.id },
    data: { createdAt: h(age), reminderCount: Number(ETAPE) - 1, lastReminderSentAt: null },
  });
  console.log("prêt pour le rappel", ETAPE, ":", u.email, p.createdAt, p.reminderCount);
  process.exit(0);
})();
```

Déclencher.

```
// scripts/recette-onboard.ts
import { processOnboardingReminders } from "../apps/auth-service/src/cron/onboarding-reminder.cron";
(async () => { console.log(await processOnboardingReminders()); process.exit(0); })();
```

CRON-ONBOARD-1 — Les trois rappels partent dans l'ordre

Étapes — pour chacune des trois étapes : vider Mailpit, préparer le compte avec le script, lancer `recette-onboard.ts`.

Résultat attendu

- Le script renvoie `{ sent: 1 }` à chaque fois.
- Un email dans Mailpit, dans la langue du destinataire, avec l'objet correspondant à l'étape.
- `CarrierPage.reminderCount` passe à 1, puis 2, puis 3 ; `lastReminderSentAt` est renseigné à chaque fois.
- Après le troisième, un quatrième passage ne renvoie plus rien : `reminderCount >= MAX_REMINDERS (3)`.

Verdict : ☐ conforme ☐ non conforme

CRON-ONBOARD-2 — Les cas où le rappel ne doit PAS partir

Cas	Préparation	Attendu
Inscription terminée	<code>carrierStatus: "ACTIVE"</code>	Aucun email — et double garde : <code>sendOnboardingReminderEmail</code> revérifie le statut avant d'envoyer
Compte effacé	<code>isDeleted: true</code>	Aucun email (règle D35)
Adresse en liste de suppression	<code>emailSuppressedAt</code> renseigné	Aucun email (règle D35)
Trop tôt	<code>CarrierPage.createdAt = now - 12 h</code> (délai du rappel 1 : 24 h)	Aucun email
Rappel récent	<code>lastReminderSentAt = now - 6 h</code>	Aucun email : intervalle minimal de 12 h
Trois rappels déjà envoyés	<code>reminderCount: 3</code>	Aucun email : la sélection ne le charge même pas

Les deux cas D35 (`isDeleted`, `emailSuppressedAt`) sont les plus importants : ils sont doublement filtrés, dans la requête Prisma **et** dans la règle pure. Un nouveau flux d'email qui oublierait ce filtre serait un bug — c'est une règle générale de la plateforme, pas une particularité de ce cron.

Verdict : ☐ conforme ☐ non conforme

CRON-ONBOARD-3 — Le compte abandonné n'est pas réveillé

Objectif	Vérifier la borne des 30 jours
----------	--------------------------------

Étapes — poser `CarrierPage.createdAt = now - 60 jours`, `reminderCount: 0`, lancer la tâche.

Résultat attendu — **aucun** email. Le compte est éligible sur tous les autres critères (statut `ONBOARDING`, zéro rappel envoyé, délai largement dépassé), mais son âge dépasse la borne. C'est exactement le scénario qui aurait été catastrophique au branchement du cron.

Verdict : ☐ conforme ☐ non conforme

CRON-ONBOARD-4 — Non-régression : pas de double rappel

Étapes — après un rappel envoyé, relancer `recette-onboard.ts` trois fois de suite, sans rien modifier.

Résultat attendu

- `{ sent: 0 }` à chaque fois : l'intervalle minimal de 12 heures s'applique.
- Un seul email dans Mailpit.
- `reminderCount` inchangé.

Limite à consigner. Contrairement à la relance des messages non lus, il n'y a **pas** de réclamation par verrou optimiste ici : le compteur est mis à jour **après** l'envoi. Deux instances de l'auth-service qui passeraient exactement au même moment pourraient donc envoyer deux rappels. La protection actuelle est le drapeau `running` (mono-instance) et l'intervalle de 12 heures (qui rattrape le second passage d'après). À signaler comme **majeure** si le déploiement cible est multi-instances.

Verdict : ☐ conforme ☐ non conforme

CRON-ONBOARD-5 — Le cron démarre bien

Objectif	Le scénario qui aurait détecté le défaut historique
Gravité si écart	Majeure

Étapes

1. Démarrer l'auth-service et lire ses premières lignes de journal.
2. Attendre le passage de l'heure ronde suivante.
3. Lire les battements.

Résultat attendu

- Au démarrage : `[auth-service] Onboarding reminder cron started (hourly)`.
- Après l'heure ronde : un battement `auth-service:onboarding-reminder`, `ok: true`, `schedule: "0 * * * *`", avec un résumé du type « 0 rappel(s) ».

C'est le test qui manquait. Un cron défini, testé unitairement, mais jamais appelé, passait toutes les vérifications sauf celle-ci : **il ne battait pas**.

Verdict : ☐ conforme ☐ non conforme

5. Relais d'événements

Le relais est le maillon le plus silencieux de toute la plateforme. Il n'a pas d'écran, pas d'horaire, pas de battement : c'est une boucle. S'il s'arrête, l'application continue de répondre correctement à tout le monde, et **rien ne part**.

5.1 Ce qu'il faut savoir avant de tester

Il y a **deux** relais, un par domaine, avec le même code de discipline :

Relais	Fichier	Type d'agrégat drainé	Sujet	Cadence	Bail
Réservations	apps/deal-service/src/relay/outbox-relay.ts	booking	booking-events	1 s	document RelayLease d'identifiant outbox-relay , TTL 30 s
Messagerie	apps/message-service/src/relay/messaging-relay.ts	conversation	messaging-events	2 s	document RelayLease d'identifiant messaging-relay , TTL 15 s

Les garanties tenues, et donc à vérifier :

1. **Écriture dans la même transaction.** Aucun changement d'état sans un événement d'outbox écrit dans la **même transaction Mongo**. Si la transaction échoue, ni l'un ni l'autre n'existe.
2. **Au moins une fois.** `publishedAt` est posé **après** l'accusé du courtier, message par message. Un arrêt entre l'accusé et l'écriture provoque une republication — c'est le consommateur qui dédoublonne.
3. **Ordre par agrégat.** Le lot est trié par `occurredAt` croissant, publié **séquentiellement**, avec `aggregateId` comme clé de partition, et **un seul relais actif** grâce au bail.
4. **Validation au contrat avant publication.** Le payload passe `BookingDomainEventSchema` (ou `MessagingDomainEventSchema`) **avant** le courtier. Un écart entre l'écrivain et le contrat est attrapé ici, jamais découvert chez un consommateur.
5. **Parking.** Un payload hors contrat ou une erreur non rejouable incrémente `attempts` ; à **10** tentatives la ligne est **parquée** : exclue de la requête, jamais supprimée.
6. **Une panne de courtier n'incrémente jamais `attempts`.** C'est une subtilité coûteuse : `kafka.js` marque `retriable: false` une simple panne de connexion une fois ses propres tentatives épuisées (`KafkaJSNumberOfRetriesExceeded`, `KafkaJSConnectionError`). Sans l'exclusion explicite de ces deux noms, une panne de dix minutes parquerait des dizaines d'événements parfaitement sains.
7. **Connexion paresseuse.** Le relais ne se connecte au courtier qu'au premier tick où il détient le bail : l'API démarre même si Redpanda est éteint.

5.2 Outils de ce chapitre

```
# Voir les deux baux
npx tsx --env-file=.env -e 'import p from "../packages/libs/prisma"; p.relayLease.findMany().then(r=>
{console.log(r);process.exit(0)})'
```

```
# Suivre le sujet en direct
docker exec yamba-redpanda rpk topic consume booking-events -f '%k | %h | %v\n'
```

```
// scripts/recette-relais-etat.ts
import prisma from "../packages/libs/prisma";
(async () => {
  const absent = { OR: [{ publishedAt: null }, { publishedAt: { isSet: false } }] } as never;
  console.log("baux :", await prisma.relayLease.findMany());
  console.log("non publiés :", await prisma.outboxEvent.count({ where: absent }));
  console.log("parqués :", await prisma.outboxEvent.count({ where: { ...(absent as object),
attempts: { gte: 10 } } as never }));
  console.log(await prisma.outboxEvent.findMany({ where: absent, orderBy: { occurredAt: "asc" },
take: 5,
  select: { id: true, aggregateType: true, eventType: true, occurredAt: true, attempts: true,
lastError: true } }));
  process.exit(0);
})();
```

CRON-RELAIS-1 — L'événement écrit dans la transaction est publié

Objectif	Le cas nominal de bout en bout : une transition métier devient un message sur le courtier
-----------------	---

Étapes

1. Vider l'outbox des lignes de recette (`seed-outbox.ts` remet les siennes à zéro).
2. Démarrer le deal-service avec le relais actif ; suivre le sujet en direct dans un second terminal.
3. Provoquer une transition réelle — le plus simple : `recette-expire.ts` sur une demande périmée (deux événements d'un coup).

Résultat attendu

- Les deux événements apparaissent en base **immédiatement** après la transition, avec `publishedAt` absent ou nul.
- Une seconde plus tard au plus, deux messages arrivent sur `booking-events`.
- Chaque message porte : une **clé** égale à l' `aggregateId` (l'identifiant de la réservation) et un **en-tête** `event-id` égal à l'identifiant Mongo de la ligne d'outbox.
- En base, `publishedAt` est maintenant renseigné sur les deux lignes.
- Le journal du deal-service affiche `Event published` avec `eventId`, `eventType`, `aggregateId`, `correlationId`.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-2 — L'ordre par agrégat est respecté

Objectif	Vérifier que cinq événements d'une même réservation arrivent dans l'ordre de leur survenue
-----------------	--

Étapes

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-outbox.ts
docker exec yamba-redpanda rpk topic consume booking-events -n 6 -o -6
```

Résultat attendu

- Six messages : cinq pour un même agrégat (`booking.requested`, `accepted`, `picked_up`, `delivered`, `completed`), un pour un second agrégat.
- Les cinq du premier agrégat arrivent **exactement dans cet ordre** — c'est l'ordre de leurs `occurredAt`, étagés d'une seconde par le script.
- Tous les cinq portent la **même clé** : ils vont donc dans la **même partition**, ce qui est la condition de l'ordre.

Trois mécanismes concourent à ce résultat : le tri par `occurredAt` dans la requête, la publication séquentielle (pas de `Promise.all`), et le bail qui garantit un unique publieur. Casser n'importe lequel des trois casse l'ordre.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-3 — Le bail d'exclusivité entre deux instances

Objectif	Deux deal-services démarrés : un seul draine
----------	--

Étapes

1. Construire : `npx nx build deal-service`.
2. Démarrer une première instance : depuis `apps/deal-service`, `PORT=6003 node --env-file=../../.env dist/main.js`.
3. Démarrer une seconde sur un autre port : `PORT=6013 node --env-file=../../.env dist/main.js`.
4. Injecter des événements (`seed-outbox.ts`) et suivre les deux journaux.
5. Lire le document `RelayLease` d'identifiant `outbox-relay`.

Résultat attendu

- Les deux instances écrivent `Outbox relay starting` avec un owner **différent** (`hostname#pid#uuid`).
- Une seule des deux écrit `Broker connection established` puis les lignes `Event published`.
- Le document `RelayLease` porte le owner de l'instance active et un `expiresAt` **renouvelé à chaque tick** (donc toujours entre maintenant et maintenant + 30 s).
- **Aucun message n'est publié deux fois** : compter les messages sur le sujet, il doit y en avoir exactement autant que de lignes d'outbox.

Étape complémentaire — la reprise. Arrêter brutalement l'instance détentrice (`kill -9`). Sans arrêt propre, le bail n'est pas libéré : la seconde instance doit reprendre le drainage **au plus tard 30 secondes après** (le TTL du bail), et écrire à son tour `Broker connection established`.

Étape complémentaire — l'arrêt propre. Arrêter l'instance détentrice par `SIGTERM` (Ctrl+C). Le journal doit montrer `Outbox relay stopped`, le bail est repoussé dans le passé (`expiresAt` à l'époque zéro), et la seconde instance reprend **immédiatement**, sans attendre les 30 secondes.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-4 — Chaque relais ne draine que son domaine

Objectif	Le cloisonnement par <code>aggregateType</code> , sans lequel chaque relais empoisonnerait les événements de l'autre
Gravité si écart	Majeure

Étapes

1. Démarrer les deux services (deal et message) avec leurs relais actifs.
2. Provoquer un événement de chaque domaine : une transition de réservation (type `booking`) et un message dans une conversation (type `conversation`), par l'écran, cahier 01).
3. Suivre les deux sujets.

Résultat attendu

- L'événement `booking.*` arrive sur `booking-events` **et nulle part ailleurs**.
- L'événement `conversation.*` arrive sur `messaging-events` **et nulle part ailleurs**.
- **Aucune** ligne `attempts` incrémentée : ni l'un ni l'autre n'a essayé de valider un événement de l'autre domaine contre son propre contrat.
- Le journal du deal-service ne mentionne jamais un `conversation.*`, et réciproquement.

C'est un scénario de non-régression au sens strict : avant le cloisonnement, le relais du deal-service lisait **tous** les événements non publiés, tentait de valider un `conversation.message_posted` contre le schéma des réservations, échouait avec une erreur Zod — donc un **poison** — et finissait par le parquer. Et réciproquement.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-5 — L'événement empoisonné est parké, jamais supprimé

Objectif	Vérifier la mécanique de parking et le compteur <code>attempts</code>
----------	---

Étapes

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-outbox.ts --with-poison
```

Puis laisser tourner le relais une vingtaine de secondes (un tick par seconde) et suivre la ligne au payload `{ poison: true }`.

Résultat attendu

- Les six événements sains sont publiés normalement — **le poison ne bloque pas la suite du lot**.
- La ligne empoisonnée voit `attempts` monter de 1 à 10, avec `lastError` (une erreur Zod) et `lastErrorAt` mis à jour à chaque tick.
- Aux tentatives 1 à 9 : `Poison event, retrying next tick`.
- À la dixième : `Poison event PARKED - manual investigation required`.
- Ensuite, la ligne **n'est plus lue** : `attempts` reste à 10, `lastError` ne bouge plus.
- La ligne **existe toujours** en base, avec sa charge utile complète.
- Le compteur `parked` de la page « État des services » vaut au moins 1, avec `parkedThreshold: 10`.
- L'alerte `OUTBOX_PARKED` (critique) apparaît sur l'accueil du back-office et sur `/alerts`, et le cron d'alertes envoie l'email au support (voir CRON-ALERTE-1).

Nettoyage — supprimer la ligne de recette :

```
npx tsx --env-file=.env -e 'import p from "./packages/libs/prisma"; p.outboxEvent.deleteMany({where: {correlationId:"seed-outbox"}}).then(r=>{console.log(r);process.exit(0)})'
```

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-6 — Le sujet absent parce des événements sains

Objectif	Éprouver le scénario d'installation le plus probable : <code>messaging-events</code> n'a pas été créé
Gravité si écart	Majeure

Étapes

1. Supprimer le sujet : `docker exec yamba-redpanda rpk topic delete messaging-events`.
2. Démarrer le message-service avec son relais actif.
3. Envoyer un message dans une conversation (cahier 01).
4. Observer la ligne d'outbox correspondante pendant une minute.

Résultat attendu à documenter précisément. L'auto-crédation de sujets étant désactivée au niveau du cluster et du producteur, la publication échoue. La question de recette est : **cet échec est-il traité comme transitoire (pas de parking) ou comme non rejouable (parking) ?**

- Si l'erreur remontée par `kafka.js` est une erreur de connexion, `attempts` **ne bouge pas** et le tick part en attente croissante (1 s → 30 s). Comportement souhaitable.
- Si elle est marquée `retriable: false` sans porter l'un des deux noms exclus, `attempts` **monte** et l'événement finit **parqué** au bout de dix tentatives, soit une vingtaine de secondes. Comportement à consigner comme anomalie majeure : un événement parfaitement sain, perdu pour une erreur d'installation.

Consigner le comportement observé, avec la valeur de `lastError`. Puis recréer le sujet (§ 2.2) et vérifier que les événements en attente partent bien — **s'ils n'ont pas été parqués**.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-7 — Le courtier redémarre en cours de route

Objectif	Vérifier qu'une panne de courtier n'entraîne aucune perte et aucun parking
Gravité si écart	Bloquante

Étapes

1. Relais actif, outbox vide de non publiés.
2. Arrêter le courtier : `docker stop yamba-redpanda`.
3. Provoquer **cinq** transitions métier (cinq expirations, par exemple) — donc dix événements d'outbox.
4. Observer le journal du deal-service pendant deux minutes.
5. Redémarrer : `docker start yamba-redpanda` et attendre le retour à l'état sain.

Résultat attendu

- Pendant la panne : l'API répond normalement, les transitions s'écrivent, les événements s'accumulent avec `publishedAt` absent.
- Le journal affiche `Relay tick failed – backing off` avec un `nextRetryMs` qui **double** à chaque échec : 1 s, 2 s, 4 s, 8 s, 16 s, puis plafond à **30 s**.
- **`attempts` reste à 0 sur les dix événements.** C'est le point capital : une panne de courtier ne parque rien. Le vérifier explicitement avec `recette-relais-etat.ts`.
- `lastError` est renseigné (trace), mais pas `attempts`.
- Après le redémarrage : au tick suivant, `Broker connection established`, puis les dix événements sont publiés **dans l'ordre de leurs `occurredAt`**, et l'attente revient à 1 s.
- Le compteur `unpublished` de la page « État des services » redescend à zéro.
- Aucun événement n'est publié deux fois : compter les messages sur le sujet.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-8 — Le relais coupé, l'application vit toujours

Étapes

```
npx nx build deal-service
cd apps/deal-service && OUTBOX_RELAY_ENABLED=false node --env-file=../../.env dist/main.js
```

Résultat attendu

- Au démarrage : Outbox relay disabled (OUTBOX_RELAY_ENABLED=false).
- Les transitions métier fonctionnent, les événements s'écrivent, **rien n'est publié**.
- Le bail outbox-relay n'est ni créé ni renouvelé.
- Après 15 minutes, l'alerte OUTBOX_LAGGING_15MIN (critique) apparaît sur l'accueil du back-office : c'est le signal attendu. Une instance API pure est un usage légitime, mais si **toutes** les instances ont le relais coupé, l'alerte doit se lever.

Verdict : ☐ conforme ☐ non conforme

CRON-RELAIS-9 — Aucun secret ne sort dans un payload

Objectif	Vérifier que le code de livraison et les coordonnées du destinataire ne quittent jamais la base
Gravité si écart	Bloquante

Étapes

1. Produire une réservation complète, du dépôt à la remise, avec un code de livraison et un destinataire renseignés (recipient.firstName, lastName, phoneE164, email).
2. Consommer **tous** les messages des deux sujets et les écrire dans un fichier :

```
docker exec yamba-redpanda rpk topic consume booking-events -o start -n 200 > /tmp/booking-events.txt
docker exec yamba-redpanda rpk topic consume messaging-events -o start -n 200 > /tmp/messaging-events.txt
```

3. Chercher les termes interdits :

```
grep -Ei 'deliveryCode|deliveryCodeHash|deliveryCodeEncrypted|742891|recipient|phoneE164|@' /tmp/booking-events.txt
grep -Ei 'deliveryCode|742891|recipient(FirstName|LastName|Phone|Email)|phoneE164' /tmp/messaging-events.txt
```

4. Refaire la même recherche dans les charges utiles stockées en base :

```
// scripts/recette-payload-audit.ts
import prisma from "../packages/libs/prisma";
const interdits = /deliveryCode|742891|phoneE164|recipientFirstName|recipientLastName/i;
(async () => {
  const rows = await prisma.outboxEvent.findMany({ select: { id: true, eventType: true, payload: true } });
  const fautifs = rows.filter((r) => interdits.test(JSON.stringify(r.payload)));
  console.log(rows.length, "événements analysés,", fautifs.length, "fautif(s)");
  for (const f of fautifs) console.log("!!", f.id, f.eventType);
  process.exit(0);
})();
```

Résultat attendu

- **Zéro** occurrence de `deliveryCode`, `deliveryCodeHash`, `deliveryCodeEncrypted` ou de la valeur `742891`.
- **Zéro** coordonnée du destinataire : ni prénom, ni nom, ni téléphone, ni adresse électronique.
- Le seul champ `recipientId` autorisé est celui des événements de messagerie : c'est un **identifiant de membre**, pas un tiers destinataire — la confusion de noms est un piège, il faut lire le contexte.
- Les charges utiles contiennent en revanche, et c'est voulu : identifiants (réservation, trajet, expéditeur, voyageur), corridor (villes et codes pays), catégorie, poids, montants en centimes, devise, acteur, jalons. Ces données sont **riches par décision** : l'historique doit rester exploitable.
- Le champ `preview` d'un `conversation.message_posted` est borné à **140 caractères** par le contrat : un extrait, jamais le fil entier.

Verdict : ☐ conforme ☐ non conforme

6. Consommateurs

Le notification-service porte **deux** consommateurs, dans le même processus, coupés ensemble par `NOTIFICATION_CONSUMER_ENABLED=false` :

Consommateur	Sujet	Groupe	Produit
<code>booking-events.consumer.ts</code>	<code>booking-events</code>	<code>notification-service</code>	Notifications in-app (matrice A15) + emails + mesure d'audience
<code>messaging-events.consumer.ts</code>	<code>messaging-events</code>	<code>messaging-notifications</code>	Notification in-app à l'autre partie + mesure d'audience

La discipline commune, en quatre temps (« réclamer d'abord ») :

1. **Réclamer.** Création d'une ligne `ConsumedEvent` en `PENDING`. La clé unique `[consumerGroup, eventId]` **fait le verrou**. Une violation d'unicité (`P2002`) signifie que quelqu'un a déjà réclamé : si l'état est `PROCESSED`, c'est un doublon, on passe ; si c'est `PENDING` ou `FAILED`, un traitement antérieur s'est interrompu, on retraite.
2. **Analyser au contrat réel.** Un échec ici est **définitif** : la ligne passe `FAILED` avec son `lastError`, et **la partition continue d'avancer**. Un message malformé ne doit jamais bloquer la file des autres — le rejeu se fait depuis l'outbox Mongo, qui est la source de vérité.
3. **Matérialiser.** `upsert` sur la clé unique `[eventId, userId]` : un événement donne N notifications, et le rejouer n'en crée pas une de plus. Puis les emails, chacun réclamé par un `EmailDelivery` unique sur `[eventId, userId]`.
4. **Marquer PROCESSED.** Avec `processedAt` et `lastError` remis à nul.

Toute erreur **transitoire** (base injoignable) est **relancée** : l'offset n'est pas validé, le courtier relivre. C'est l'« au moins une fois » assumé.

Deux détails à connaître pour la recette :

- **fromBeginning: true** à la souscription : au **tout premier** démarrage d'un groupe (aucun offset validé), le consommateur lit depuis le début du sujet. Dès le premier offset validé, ce drapeau n'a plus d'effet.
- Un message **sans en-tête event-id** est ignoré avec une ligne d'erreur : pas de clé d'idempotence, donc pas de traitement sûr possible.

6.1 Outils de ce chapitre

```
docker exec yamba-redpanda rpk group describe notification-service
docker exec yamba-redpanda rpk group describe messaging-notifications
```

```
// scripts/recette-conso-etat.ts - usage : ... [eventId]
import prisma from "../packages/libs/prisma";
const ID = process.argv[2];
(async () => {
  if (ID) {
    console.log("consommations :", await prisma.consumedEvent.findMany({ where: { eventId: ID } }));
    console.log("notifications :", await prisma.notification.findMany({ where: { eventId: ID },
select: { id: true, userId: true, type: true, readAt: true } }));
    console.log("emails :", await prisma.emailDelivery.findMany({ where: { eventId: ID }, select: {
id: true, userId: true, template: true, status: true, sentAt: true } }));
  } else {
    console.log(await prisma.consumedEvent.groupBy({ by: ["consumerGroup", "status"], _count: true
}));
  }
  process.exit(0);
})();
```

```
// scripts/recette-rejouer.ts - usage : ... <outboxEventId>
// Remet un événement à l'état « non publié » : le relais le republiera au tick suivant.
import prisma from "../packages/libs/prisma";
(async () => {
  const r = await prisma.outboxEvent.update({ where: { id: process.argv[2] }, data: { publishedAt:
null } });
  console.log("rejeu armé :", r.id, r.eventType);
  process.exit(0);
})();
```

CRON-CONSO-1 — L'événement devient notification et email

Étapes

1. Vider Mailpit. Démarrer le relais et les consommateurs.
2. Provoquer une transition réelle — par exemple une expiration (`booking.expired` + `booking.refund_issued`).
3. Relever l'identifiant de la ligne d'outbox, puis `recette-conso-etat.ts <eventId>`.

Résultat attendu

- Une ligne `ConsumedEvent` en `PROCESSED`, groupe `notification-service`, avec `processedAt` renseigné et `lastError` nul.
- Les notifications correspondent **exactement** à la matrice : `booking.expired` → **l'Expéditeur seul** ; `booking.refund_issued` → **personne** (règle `NONE`, l'information est portée par l'expiration).
- Chaque Notification porte `readAt: null` **explicitement** (pas un champ absent) — sans quoi le compteur de non-lus serait faux.
- Les emails attendus sont dans Mailpit, un par destinataire, avec une ligne `EmailDelivery` en `SENT`.
- Le journal du `notification-service` affiche `Event materialized` avec le nombre de destinataires.

Verdict : ☒ conforme ☐ non conforme

CRON-CONSO-2 — Rejouer le même événement ne produit ni doublon d'email ni doublon de notification

Objectif	La garantie centrale de l'idempotence
Gravité si écart	Majeure

Étapes

1. Après CRON-CONSO-1, compter précisément : notifications pour cet `eventId`, emails dans Mailpit, lignes `EmailDelivery`.
2. `recette-rejouer.ts` `<outboxEventId>` : la ligne repasse à `publishedAt: null`.
3. Le relais la republie au tick suivant — le même message, avec **le même en-tête `event-id`**.
4. Recompter.

Résultat attendu

- Le journal du notification-service affiche Duplicate delivery – skipped .
- **Aucune** notification supplémentaire.
- **Aucun** email supplémentaire dans Mailpit.
- La ligne ConsumedEvent est inchangée (toujours PROCESSED , même processedAt).
- Le message, lui, apparaît **deux fois** sur le sujet — c'est normal et voulu : l'« au moins une fois » est assumé côté transport, le dédoublement est fait côté traitement.

Étape complémentaire — la ceinture et les bretelles. Supprimer la ligne `ConsumedEvent` puis rejouer. Le consommateur retraite. Attendu : **toujours pas** de notification en double (l'upsert sur `[eventId, userId]` la retrouve) ; en revanche l'email **peut** repartir si sa ligne `EmailDelivery` a aussi été supprimée. Consigner le comportement observé : c'est la même mécanique que CRON-CONSERV-4.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-3 — Le message malformé ne bloque pas la file

Objectif	Un message hors contrat est traité comme définitivement perdu, sans arrêter les suivants
-----------------	--

Étapes

1. Publier directement un message hors contrat sur le sujet, avec un en-tête `event-id` fabriqué : `sh docker exec -i yamba-redpanda rpk topic produce booking-events \ -H event-id=000000000000000000000099 -k test <<< '{"pas":"un evenement"}'`
2. Publier ensuite un message valide (ou provoquer une transition réelle).

Résultat attendu

- Le journal affiche `Event FAILED (definitive) – partition kept flowing.`
- Une ligne `ConsumedEvent` en **FAILED** avec son `lastError`.
- Aucune notification, aucun email pour cet événement.
- **Le message suivant est traité normalement** : la partition n'est pas bloquée.
- Le retard du groupe (`rpk group describe`) retombe à zéro.

Si la partition se bloquait ici, une seule ligne malformée arrêterait toutes les notifications de la plateforme. C'est le scénario à ne jamais laisser régresser.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-4 — Le message sans en-tête `event-id` est ignoré

Étapes

```
docker exec -i yamba-redpanda rpk topic produce booking-events -k test <<< '{"peu":"importe"}'
```

Résultat attendu — le journal affiche `Message without event-id header - skipped`, avec le sujet, la partition et l'offset. Aucune ligne `ConsumedEvent`, aucune notification, et **l'offset est validé** : le message ne sera pas relivré en boucle.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-5 — Panne du consommateur et rattrapage

Objectif	Vérifier que rien n'est perdu pendant l'arrêt et que le retard se résorbe
-----------------	---

Étapes

1. Arrêter le notification-service.
2. Provoquer **cinq** transitions métier — le relais publie normalement (il est dans un autre service).
3. Constater le retard : `docker exec yamba-redpanda rpk group describe notification-service` → la colonne LAG doit valoir le nombre de messages publiés.
4. Redémarrer le notification-service.

Résultat attendu

- Pendant l'arrêt : les messages sont sur le sujet, aucune notification, aucun email.
- Au redémarrage : `Consumer running` puis le traitement des cinq événements en rafale.
- Le retard retombe à **0**.
- Les notifications et les emails arrivent **tous**, une seule fois chacun.
- L'ordre de traitement par agrégat est respecté (les événements d'une même réservation dans l'ordre de leur survenue).

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-6 — La panne de base ne valide pas l'offset

Objectif	Vérifier que l'erreur transitoire provoque bien une relivraison, et non une perte silencieuse
-----------------	---

Étapes — c'est le scénario le plus délicat à monter. Deux approches :

(a) *Par la coupure réseau.* Rendre Mongo injoignable (couper le réseau, ou pointer `DATABASE_URL` sur un hôte inexistant) pendant qu'un message arrive.

(b) *Par l'observation d'un incident réel.* Si une coupure Atlas se produit pendant la campagne, relever le comportement.

Résultat attendu

- L'erreur remonte, l'offset n'est **pas** validé.

- Le journal montre l'erreur, pas un `Event materialized`.
- Une fois la base revenue, le message est **relivré** et traité.
- La notification finit par exister, en **un** seul exemplaire.

Si ce scénario ne peut pas être monté proprement, le classer **non joué** plutôt que conforme. Une garantie non éprouvée n'est pas une garantie.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-7 — Les deux groupes sont indépendants

Étapes — `docker exec yamba-redpanda rpk group list`, puis décrire les deux.

Résultat attendu

- Exactement deux groupes : `notification-service` et `messaging-notifications`.
- Chacun n'est abonné qu'à **son** sujet.
- Les offsets de l'un n'affectent pas l'autre : arrêter un consommateur (par exemple en supprimant le sujet `messaging-events`) laisse l'autre fonctionner.
- Les deux noms sont **exactement** ceux du registre `packages/libs/messaging/src/consumer-groups.ts`. Les renommer reviendrait à perdre tous les offsets et à retraiter le sujet entier — ce serait, en production, une avalanche de notifications et d'emails en double.

Verdict : ☐ conforme ☐ non conforme

CRON-CONSO-8 — La mesure d'audience ne reçoit que la liste blanche

Objectif	Vérifier que le déversement vers PostHog ne fait pas sortir de donnée personnelle
Gravité si écart	Bloquante

Étapes

1. Sans `POSTHOG_API_KEY` : vérifier que le déversement est **inerte** (aucun appel réseau, aucune erreur, aucun blocage du consommateur).
2. Avec une clé factice et un hôte pointant sur un serveur d'écoute local, ou en lisant simplement les tests unitaires du paquet : vérifier la forme des propriétés envoyées.

Résultat attendu

- Les seules propriétés transmises sont celles de la liste blanche : `bookingId`, `tripId`, `conversationId`, `category`, `categoryFamily`, `weightKg`, `transportCents`, `totalShipperCents`, `currencyCode`, `actor`, `status`, `reason`, `outcome`, `kind`, `amountCents`, plus le corridor (villes, codes pays).
- **Jamais** de nom, d'adresse électronique, de téléphone, d'adresse postale, de photo, ni de code de livraison.
- L'identifiant transmis est le **seul identifiant technique** du membre.
- Seuls les membres ayant `analyticsOptIn: true` reçoivent un événement : le vérifier en basculant la préférence d'un compte de test.

Verdict : ☐ conforme ☐ non conforme

7. Battements et moniteur externe

Ce chapitre teste le seul dispositif qui rend visible une tâche morte.

7.1 La mécanique

Chaque tâche est enveloppée par `withHeartbeat` (`packages/libs/redis/cron-heartbeat.ts`). À chaque passage :

1. le travail est exécuté ;
2. un enregistrement JSON est écrit dans Redis sous `yamba:cron:<service>:<nom>`, TTL **7 jours** ;
3. **si le passage a réussi**, un `GET` best effort part vers l'adresse déclarée pour ce cron dans `CRON_HEARTBEAT_PING_URLS` (délai maximal 3 s) ;
4. en cas d'échec, l'enregistrement porte `ok: false` et le message d'erreur, l'exception est relancée vers l'appelant — **et aucun battement sortant ne part**.

Trois propriétés à retenir :

- **Le battement est best effort.** Un Redis absent ne fait jamais échouer une tâche : `recordCronRun` avale ses propres exceptions. C'est voulu — un versement ne doit pas rater parce que Redis a hoqueté.
- **Le battement sortant ne part qu'après un succès.** C'est ce qui permet à un moniteur externe de conclure : « pas de signal depuis deux périodes » signifie « la tâche ne tourne plus, ou elle échoue ».
- **Le TTL de 7 jours** fait disparaître la clé d'une tâche morte. Une clé absente est donc soit « jamais passée », soit « morte depuis plus d'une semaine ».

7.2 La carte des battements sortants

`CRON_HEARTBEAT_PING_URLS` est un objet JSON dont les clés sont `"<service>:<cron>"` :

```
CRON_HEARTBEAT_PING_URLS={"deal-service:payout-bookings":"https://uptime.betterstack.com/api/v1/heartbeat/xxxx","deal-service:expire-bookings":"https://.../yyyy","message-service:unread-reminder":"https://.../zzzz","deal-service:ops-alerts":"https://.../www"}
```

Les quatre battements du runbook, avec leur période de surveillance conseillée :

Clé	Horaire du cron	Période à déclarer chez le moniteur
<code>deal-service:payout-bookings</code>	toutes les 5 min	2 h
<code>deal-service:expire-bookings</code>	toutes les 5 min	15 min
<code>message-service:unread-reminder</code>	toutes les 5 min	15 min
<code>deal-service:ops-alerts</code>	toutes les heures	2 h

Un JSON invalide ne provoque **aucune** erreur : la fonction de résolution avale l'exception et ne renvoie aucune adresse. Autrement dit, **une faute de frappe dans cette variable désarme silencieusement la surveillance**. C'est le scénario CRON-BATT-5.

CRON-BATT-1 — Chaque tâche laisse sa trace

Objectif	Les treize tâches battent
----------	---------------------------

Étapes

1. Démarrer les six services.

2. Attendre au moins un cycle complet — pour les nocturnes, **forcer** un passage avec les scripts du § 4 (le battement n'est posé que par `withHeartbeat`, donc par le cron lui-même : appeler la fonction de travail directement **ne pose pas** de battement).
3. Lancer `recette-battements.ts`.

Résultat attendu — treize entrées, chacune avec :

- `service` et `name` conformes au tableau du § 3.5 ;
- `schedule` égal à l'expression cron du code ;
- `ok: true` ;
- `ranAt` cohérent avec l'horaire ;
- `durationMs` plausible (quelques dizaines à quelques centaines de millisecondes sur un jeu d'essai) ;
- un `summary` lisible, sauf pour `payout-bookings` qui n'en produit pas (`runPayoutPasses` ne renvoie rien) — ce n'est pas une anomalie, c'est une limite à connaître : ce battement dit « je suis passé », pas « j'ai fait ceci ».

Vérification croisée — les mêmes treize entrées apparaissent sur `http://localhost:3001/status`, bloc des tâches planifiées.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-2 — Une tâche coupée ne bat plus

Objectif	Le scénario qui rend une panne visible
----------	--

Étapes

1. Relever l'horodatage du battement `deal-service:expire-bookings`.
2. Redémarrer le `deal-service` avec `BOOKING_EXPIRY_CRON_ENABLED=false` (par le paquet construit, § 2.6).
3. Attendre **quinze minutes** (trois périodes de cinq).
4. Relire les battements.

Résultat attendu

- L'horodatage de `deal-service:expire-bookings` **n'a pas bougé**.
- Les autres battements du même service, eux, ont avancé — ce qui prouve que le service tourne et que seule cette tâche est coupée.
- Sur la page « État des services », le battement affiche un âge de quinze minutes pour un horaire de cinq minutes : c'est le signal.
- Après sept jours, la clé disparaîtrait complètement (TTL). Ce cas ne se joue pas en campagne, il se documente.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-3 — Une tâche en échec bat quand même, avec son erreur

Étapes

1. Provoquer un échec durable : par exemple arrêter Mailpit et forcer le passage du récapitulatif quotidien (dont l'erreur d'envoi n'est pas avalée, voir CRON-DIGEST-4).
2. Relire les battements.

Résultat attendu

- Le battement `deal-service:ops-digest` existe, avec `ok: false`, `summary: null` et un `error` non vide.

- `durationMs` est renseigné.
- Sur la page « État des services », la ligne est signalée en erreur.
- **Aucun battement sortant** n'est parti vers le moniteur externe pour ce passage : le moniteur verra donc un signal manquant, ce qui est le comportement voulu.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-4 — L'adresse de battement sortante est appelée

Objectif	Vérifier que le <code>GET</code> best effort part réellement après un passage réussi
Prérequis	Un point d'écoute HTTP local

Étapes

1. Ouvrir un serveur d'écoute simple : `sh python3 -m http.server 9999`
2. Déclarer un battement sortant dans `.env` : `CRON_HEARTBEAT_PING_URLS={"deal-service:expire-bookings":"http://localhost:9999/battement-expire"}`
3. Redémarrer le `deal-service`.
4. Attendre un tick de cinq minutes.

Résultat attendu

- Le serveur d'écoute journalise une requête `GET /battement-expire` (réponse 404, sans importance : le battement est best effort).
- La requête arrive **après** un passage réussi, pas avant.
- Un passage **en échec** ne produit **aucune** requête.
- Le point d'écoute arrêté, le cron continue de fonctionner normalement : le délai de 3 s et l'absorption des exceptions garantissent qu'un moniteur en panne ne casse jamais une tâche. **Le vérifier explicitement** : arrêter le serveur d'écoute et constater que les passages suivants réussissent toujours.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-5 — Une carte de battements invalide désarme la surveillance en silence

Objectif	Documenter le piège d'exploitation le plus probable
Gravité si écart	Mineure (comportement attendu), mais à connaître absolument

Étapes

1. Poser un JSON volontairement invalide : `CRON_HEARTBEAT_PING_URLS={ceci n'est pas du JSON}`.
2. Redémarrer, attendre un tick, observer le serveur d'écoute.

Résultat attendu

- **Aucune** requête sortante.
- **Aucune** erreur dans les journaux du service.
- Les battements internes Redis continuent normalement.

Conséquence : la surveillance externe est éteinte et rien ne le dit. Le contrôle de non-régression consiste à vérifier, après tout changement de cette variable, qu'au moins **un** signal arrive chez le moniteur dans l'heure. À inscrire dans la liste de vérification de mise en production.

Vérification complémentaire — une adresse non `http(s)` (par exemple `ftp://...`) est aussi ignorée : la résolution exige le préfixe. Même conséquence.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-6 — Redis absent ne casse aucune tâche

Objectif	Vérifier que le battement est vraiment best effort
----------	--

Étapes — rendre Redis injoignable (pointer `REDIS_DATABASE_URI` sur un port fermé), démarrer le deal-service, forcer un passage.

Résultat attendu

- La tâche **fait son travail** : les réservations expirent, les événements sont écrits.
- Aucun battement n'est enregistré (Redis est absent).
- La tâche ne lève pas d'exception à cause du battement.
- La page « État des services » affiche un service `degraded` sur son contrôle Redis, et une liste de battements vide (`listCronRuns` renvoie une liste vide en cas d'erreur, sans faire échouer la page).

Ce scénario est important : il prouve que le dispositif d'observation ne peut pas devenir lui-même une cause de panne.

Verdict : ☐ conforme ☐ non conforme

CRON-BATT-7 — La page « État des services » dit la vérité

Étapes — ouvrir `http://localhost:3001/status` avec un compte OPS ou SUPER_ADMIN, et comparer chaque bloc à une mesure indépendante.

Bloc de la page	Mesure indépendante
Services (six lignes)	<code>curl http://localhost:600x/health</code> pour chacun, et <code>/gateway-health</code> pour le gateway
Tâches planifiées	<code>recette-battements.ts</code>
Outbox : <code>unpublished</code> , <code>oldestUnpublishedAt</code> , <code>parked</code> , <code>parkedThreshold</code>	<code>recette-outbox.ts</code>
Emails : <code>failedLast24h</code> , <code>sentLast24h</code>	Comptage direct de <code>EmailDelivery</code> sur 24 h
Maintenance	<code>GET /admin/maintenance</code>

Résultat attendu — les chiffres coïncident exactement. La page envoie `Cache-Control: no-store` : un rafraîchissement doit donner des valeurs fraîches, jamais une copie de cache.

Verdict : ☐ conforme ☐ non conforme

8. Sécurité et conformité

Ce chapitre reprend les exigences non négociables qui traversent tout ce cahier. Chaque écart y est **bloquant** par défaut.

CRON-SEC-1 — Une purge ne supprime jamais un événement non publié

Règle	Un événement d'outbox jamais publié — donc parqué — est une piste d'audit. Il n'est jamais supprimé, quelle que soit son ancienneté
--------------	---

Étapes — fabriquer trois lignes non publiées d'âges différents (10 jours, 200 jours, 2 ans), dont une parquée à dix tentatives, puis lancer **toutes** les purges nocturnes : `recette-purgeout.ts booking`, `recette-purgeout.ts conversation`, `recette-conservation.ts`, `recette-purgefil.ts`.

Résultat attendu — les trois lignes existent toujours, intactes, avec leurs `attempts`, `lastError` et charge utile.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-2 — Une purge ne déborde jamais de son domaine

Règle	Chaque service purge ses propres collections. Une purge qui touche une collection d'un autre service est une faute d'architecture
--------------	---

Étapes — relever les totaux de **toutes** les collections sensibles avant et après un passage de chacune des cinq tâches de suppression (`outbox-retention x2`, `conversation-retention`, `retention`, `recipient-redaction`).

Résultat attendu — chaque tâche ne fait varier que les collections de son tableau :

Tâche	Collections touchées
<code>outbox-retention (deal)</code>	<code>OutboxEvent</code> avec <code>aggregateType: "booking"</code>
<code>outbox-retention (message)</code>	<code>OutboxEvent</code> avec <code>aggregateType: "conversation"</code>
<code>conversation-retention</code>	<code>Conversation</code> , <code>Message</code> , <code>Meetup</code> , <code>PhoneReveal</code>
<code>retention (notification)</code>	<code>Notification</code> , <code>EmailDelivery</code> , <code>ConsumedEvent</code>
<code>recipient-redaction</code>	<code>Booking</code> (champ <code>recipient</code> uniquement)

Aucune de ces tâches ne doit faire varier `Booking` (hors le champ `recipient`), `Trip`, `User`, `Dispute`, `Review` ou `Report`.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-3 — Le signalement survit à la purge du fil

Règle	Un <code>Report</code> est un dossier de modération, pas un propos. Il survit à la disparition du message qu'il vise
--------------	--

Étapes — signaler un message (cahier 01), rendre la conversation purgeable, purger, puis ouvrir la file `/reports` du back-office.

Résultat attendu — le signalement est toujours là, avec son motif, son statut et sa date. Le corps du message a disparu avec le fil. L'administrateur voit un dossier sans contenu — c'est le compromis assumé entre conservation et modération.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-4 — L'effacement du tiers respecte exactement le délai paramétré

Règle	RGP-02 : ni avant (un litige peut avoir besoin des coordonnées), ni indéfiniment après
-------	--

Étapes

1. Le délai par défaut étant de 30 jours, préparer trois réservations terminées il y a **29, 30 et 31** jours.
2. Lancer la tâche.

Résultat attendu

- Celle de **31 jours** est effacée.
- Celle de **29 jours** ne l'est pas.
- Celle de **30 jours** dépend de l'heure exacte : la comparaison est stricte (`< now - N jours`). Consigner le résultat observé, sans en faire une anomalie tant qu'il est cohérent avec l'écart d'heures.
- Aucune réservation en cours ni en litige n'est touchée.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-5 — Aucun email ne part vers un compte effacé ou une adresse supprimée

Règle	D35 : tout résolveur de destinataire doit écarter <code>isDeleted</code> et <code>emailSuppressedAt</code> . Un nouveau flux d'email qui l'oublie est un bug
-------	--

Étapes — pour **chaque** flux d'email issu d'une tâche planifiée, préparer deux comptes de test : l'un avec `isDeleted: true`, l'autre avec `emailSuppressedAt` renseigné. Puis rendre chaque flux éligible et vider Mailpit avant chaque passage.

Flux	Tâche	Résolveur
Rappel d'inscription	<code>onboarding-reminder</code>	Filtre dans la requête Prisma et dans la règle pure
Relance des messages non lus	<code>unread-reminder</code>	Filtre dans <code>remind()</code> : <code>isDeleted</code> , <code>emailSuppressedAt</code> , <code>messagingReminderEmails</code>
Rappel de vérification J+3	<code>payout-bookings</code>	Par le consommateur d'événements et <code>dispatchBookingEmails</code>
Relance de notation	<code>rating</code>	Idem
Alertes de seuil	<code>ops-alerts</code>	Destinataire fixe (<code>SUPPORT_EMAIL</code>) — non concerné
Récapitulatif quotidien	<code>ops-digest</code>	Destinataire fixe — non concerné

Résultat attendu — **zéro** email dans Mailpit pour les deux comptes de test, sur tous les flux concernés. Un compte effacé garde une adresse technique non vide (`erased+<id>@anonymised.invalid`) : c'est précisément pour cela que le filtre doit porter sur `isDeleted`, jamais sur « l'adresse est vide ».

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-6 — Le code de livraison ne quitte jamais la base

Règle	Le code ne voyage ni dans un événement, ni dans un email, ni dans une notification, ni dans la mesure d'audience
-------	--

Étapes — reprendre l'audit de CRON-RELAIS-9, et l'étendre aux trois autres surfaces :


```
// scripts/recette-secret-audit.ts
import prisma from "../packages/libs/prisma";
const interdit = /deliveryCode|742891/i;
(async () => {
  const notifs = await prisma.notification.findMany({ select: { id: true, type: true, payload: true } });
  const outbox = await prisma.outboxEvent.findMany({ select: { id: true, eventType: true, payload: true } });
  const faux = [
    ...notifs.filter((n) => interdit.test(JSON.stringify(n.payload))).map((n) => ["notification", n.id, n.type]),
    ...outbox.filter((o) => interdit.test(JSON.stringify(o.payload))).map((o) => ["outbox", o.id, o.eventType]),
  ];
  console.log(notifs.length + outbox.length, "documents analysés,", faux.length, "fautif(s)");
  for (const f of faux) console.log("!!", ...f);
  process.exit(0);
})();
```

Et, côté emails : dans Mailpit, chercher 742891 dans le corps de **tous** les messages reçus pendant la campagne.

Résultat attendu — zéro occurrence partout. En base, le code n'existe que sous deux formes : `deliveryCodeHash` (empreinte bcrypt, pour la validation) et `deliveryCodeEncrypted` (chiffré AES-256-GCM, pour le seul réaffichage à l'Expéditeur). Ni l'un ni l'autre n'apparaît dans une vue Voyageur, une liste, un événement ou un email.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-7 — Les tâches ne contournent jamais les machines à états

Règle	Une tâche planifiée est un acteur comme un autre (<code>SYSTEM</code>) : elle passe par la machine à états, elle ne fait pas ses propres <code>if</code>
-------	---

Étapes — pour chaque tâche qui change un statut (`complete-trips` , `expire-bookings` , `payout-bookings`), fabriquer un cas où la machine doit refuser et vérifier que le refus vient bien d'elle :

Tâche	Cas de refus	Garde
<code>complete-trips</code>	Trajet avec un deal en cours	<code>canPerform(trip, "complete", ctx)</code>
<code>expire-bookings</code>	Réservation non périmée	<code>onlyIfExpired</code>
<code>payout-bookings</code>	Remise avant l'échéance, ou réservation en litige	<code>onlyIfPayoutDue</code>

Résultat attendu — la tâche saute l'élément et le compte dans ses « ignorés », sans écrire en base. Un `if` recopié dans un cron qui divergerait de la machine serait une anomalie **majeure** : la règle doit avoir un seul lieu.

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-8 — Aucune écriture d'état sans son événement

Règle	Pas de changement d'état sans un événement d'outbox écrit dans la même transaction
-------	---

Étapes

1. Vider les événements de recette.

2. Provoquer, une par une, toutes les transitions produites par des tâches planifiées : expiration, complétion automatique, versement, rappel de vérification, relance de notation, révélation.
3. Pour chacune, vérifier qu'il existe au moins une ligne d'outbox correspondante, avec le bon `aggregateId` et le bon `eventType`.

Résultat attendu — la correspondance est exacte :

Transition	Événement(s) attendu(s)
Expiration	<code>booking.expired</code> et <code>booking.refund_issued</code>
Complétion automatique	<code>booking.completed</code>
Versement	<code>booking.payout_sent</code>
Rappel J+3	<code>booking.verifcation_reminder</code>
Relance de notation	<code>booking.rating_reminder</code> (un par rôle ciblé)
Révélation	<code>booking.rating_revealed</code> — sauf si personne n'a noté

Trois tâches n'émettent **aucun** événement, et c'est normal : `recipient-redaction` (effacement pur), les deux `outbox-retention` et `retention` (suppressions), `ops-digest` et `ops-alerts` (lectures et emails directs), `complete-trips` (l'état du trajet n'est pas un événement du domaine réservation).

Verdict : ☐ conforme ☐ non conforme

CRON-SEC-9 — Les montants restent des centiers entiers, jamais recalculés

Règle	Les montants sont des centimes entiers, et l'instantané de prix d'une réservation est immuable
-------	--

Étapes — après un versement automatique, comparer `payoutAmountCents` à `pricing.transportCents`, et vérifier que le prix courant du trajet n'a aucune influence : modifier le prix du `Trip` puis rejouer un versement sur une autre réservation du même trajet.

Résultat attendu — `payoutAmountCents` vaut exactement `pricing.transportCents` de la réservation, figé au moment de la réservation. Aucun nombre à virgule flottante nulle part. Un montant recalculé depuis le trajet serait une anomalie **bloquante**.

Verdict : ☐ conforme ☐ non conforme

9. Consignation

9.1 Tableau de suivi

À remplir intégralement. Un scénario sans verdict est un scénario non joué.

ID	Intitulé	Verdict	Gravité	Anomalie	Testeur	Date
CRON-TRAJETS-1	Le trajet terminé passe COMPLETED					
CRON-TRAJETS-2	Le trajet avec un deal en cours n'est pas terminé					
CRON-TRAJETS-3	Le litige ne bloque pas la complétion					
CRON-TRAJETS-4	Un trajet en échec ne bloque pas la fournée					
CRON-EXPIRE-1	La demande dépassée expire et libère tout					
CRON-EXPIRE-2	La demande non dépassée n'est pas touchée					

ID	Intitulé	Verdict	Gravité	Anomalie	Testeur	Date
CRON-EXPIRE-3	Pas de double expiration ni double restitution					
CRON-EXPIRE-4	La garde de chevauchement					
CRON-EXPIRE-5	La coupure par variable d'environnement					
CRON-PAYOUT-1	Le versement à échéance part					
CRON-PAYOUT-2	La remise non échue n'est pas versée					
CRON-PAYOUT-3	Le rappel de vérification à J+3					
CRON-PAYOUT-4	Jamais deux rappels					
CRON-PAYOUT-5	Le rejeu espacé d'un versement en échec					
CRON-PAYOUT-6	Jamais deux transferts					
CRON-PAYOUT-7	L'ordre des trois passes					
CRON-ALERTES-1	Une alerte nouvelle déclenche un email					
CRON-ALERTES-2	Rien sous le seuil, rien à envoyer					
CRON-ALERTES-3	Une alerte, un email par jour					
CRON-ALERTES-4	Le seuil est lu dans les paramètres					
CRON-ALERTES-5	Le verrou est posé même sans email					
CRON-NOTATION-1	La première relance part à J+5					
CRON-NOTATION-2	La révélation à la fin de la fenêtre					
CRON-NOTATION-3	Ce que la tâche ne doit pas faire					
CRON-NOTATION-4	Jamais deux relances ni deux révélations					
CRON-NOTATION-5	Le compteur avance même sans cible					
CRON-DIGEST-1	Le récapitulatif part avec ses trois listes					
CRON-DIGEST-2	Rien à signaler, rien à envoyer					
CRON-DIGEST-3	Un seul email par passage					
CRON-DIGEST-4	L'erreur d'envoi remonte					
CRON-RELANCE-1	Le message non lu déclenche une relance					
CRON-RELANCE-2	Les six cas de non-relance					
CRON-RELANCE-3	Le verrou optimiste empêche le double envoi					
CRON-RELANCE-4	Les délais sont lus dans les paramètres					
CRON-RELANCE-5	Le destinataire qui a coupé les relances					
CRON-PURGEFIL-1	La conversation d'un vieux deal disparaît					
CRON-PURGEFIL-2	Les trois cas de non-purge					
CRON-PURGEFIL-3	Rejouer ne casse rien					
CRON-PURGEFIL-4	La durée est lue dans les paramètres					
CRON-PURGEOUT-1	Les événements publiés et anciens partent					
CRON-PURGEOUT-2	L'événement parqué survit					
CRON-PURGEOUT-3	Chaque service ne purge que son domaine					
CRON-PURGEOUT-4	Le récent ne part pas, durée paramétrable					
CRON-PURGEOUT-5	Rejouer ne supprime rien de plus					
CRON-DESTINATAIRE-1	Le tiers est effacé après le délai					
CRON-DESTINATAIRE-2	Les trois cas de non-effacement					

ID	Intitulé	Verdict	Gravité	Anomalie	Testeur	Date
CRON-DESTINATAIRE-3	Jamais deux effacements					
CRON-DESTINATAIRE-4	Le délai est lu et borné					
CRON-CONSERV-1	Les trois collections sont purgées					
CRON-CONSERV-2	Chaque durée est indépendante					
CRON-CONSERV-3	La purge n'efface pas ce qui n'est pas à elle					
CRON-CONSERV-4	Le registre purgé rouvre le retraitement					
CRON-ONBOARD-1	Les trois rappels partent dans l'ordre					
CRON-ONBOARD-2	Les cas de non-envoi					
CRON-ONBOARD-3	Le compte abandonné n'est pas réveillé					
CRON-ONBOARD-4	Pas de double rappel					
CRON-ONBOARD-5	Le cron démarre bien					
CRON-RELAIS-1	L'événement transactionnel est publié					
CRON-RELAIS-2	L'ordre par agrégat est respecté					
CRON-RELAIS-3	Le bail d'exclusivité entre deux instances					
CRON-RELAIS-4	Chaque relais ne draine que son domaine					
CRON-RELAIS-5	L'événement empoisonné est parké					
CRON-RELAIS-6	Le sujet absent parké des événements sains					
CRON-RELAIS-7	Le courtier redémarre en cours de route					
CRON-RELAIS-8	Le relais coupé, l'application vit					
CRON-RELAIS-9	Aucun secret dans un payload					
CRON-CONSO-1	L'événement devient notification et email					
CRON-CONSO-2	Rejouer ne produit pas de doublon					
CRON-CONSO-3	Le message malformé ne bloque pas la file					
CRON-CONSO-4	Le message sans en-tête est ignoré					
CRON-CONSO-5	Panne du consommateur et rattrapage					
CRON-CONSO-6	La panne de base ne valide pas l'offset					
CRON-CONSO-7	Les deux groupes sont indépendants					
CRON-CONSO-8	La mesure d'audience sur liste blanche					
CRON-BATT-1	Chaque tâche laisse sa trace					
CRON-BATT-2	Une tâche coupée ne bat plus					
CRON-BATT-3	Une tâche en échec bat avec son erreur					
CRON-BATT-4	L'adresse de battement sortante est appelée					
CRON-BATT-5	Une carte invalide désarme la surveillance					
CRON-BATT-6	Redis absent ne casse aucune tâche					
CRON-BATT-7	La page « État des services » dit la vérité					
CRON-SEC-1	Une purge ne supprime jamais un non publié					
CRON-SEC-2	Une purge ne déborde jamais de son domaine					
CRON-SEC-3	Le signalement survit à la purge du fil					
CRON-SEC-4	L'effacement du tiers respecte le délai					
CRON-SEC-5	Aucun email vers un compte effacé					

ID	Intitulé	Verdict	Gravité	Anomalie	Testeur	Date
CRON-SEC-6	Le code de livraison ne sort jamais					
CRON-SEC-7	Les tâches passent par les machines à états					
CRON-SEC-8	Aucune écriture d'état sans son événement					
CRON-SEC-9	Montants entiers, jamais recalculés					

Total : 90 scénarios.

9.2 Fiche d'anomalie

Une anomalie par ligne « non conforme ». Format imposé :

```

ID      : ANO-CRON-<n>
Scénario : CRON-<NOM>-<n>
Gravité : bloquante | majeure | mineure | cosmétique
Nature  : fonctionnelle | documentaire

Contexte
  Branche, date, services démarrés, jeu d'essai rejoué (oui/non),
  état de Redpanda et de Mailpit, paramètres modifiés.

Reproduction
  1. ...
  2. ...
  (les commandes exactes, copiables)

Attendu   : ...
Observé   : ...

Preuves
  - Sortie du script de déclenchement
  - Extrait des journaux du service (avec l'identifiant de corrélation)
  - Battement concerné (JSON complet)
  - État de la ligne en base (avant / après)
  - Capture de la page « État des services » si pertinent

```

Ce qu'il faut joindre systématiquement, parce que ces éléments ne sont plus reconstituables après coup :

- l'**identifiant de corrélation** (`correlationId`) de l'événement en cause, qui relie l'outbox, le courtier, le consommateur et l'email ;
- l'**event-id** (identifiant Mongo de la ligne d'outbox), qui relie la notification et la trace d'email ;
- le **JSON complet du battement** au moment de l'incident (le TTL est de 7 jours : passé ce délai, la preuve n'existe plus) ;
- le **retard du groupe de consommation** (`rpk group describe`) si l'anomalie touche les notifications.

9.3 Critères de sortie

La campagne est déclarée **terminée et favorable** quand les cinq conditions sont réunies.

1. **Zéro anomalie bloquante ouverte**. En particulier : aucun double versement, aucun versement manquant, aucun secret sorti dans un payload ou un email, aucune purge débordante, aucune donnée personnelle survivant à son délai.
2. **Zéro anomalie majeure ouverte**, ou bien chaque anomalie majeure restante est accompagnée d'un contournement écrit, validé, et d'une décision portée au registre.
3. **Les treize battements présents et frais**. Après vingt-quatre heures de fonctionnement continu, la page « État des services » montre treize tâches, toutes `ok`, aucune plus vieille que deux fois son horaire.

4. **La chaîne événementielle prouvée de bout en bout**, dans les deux domaines : une transition métier produit un événement, l'événement est publié, consommé une seule fois, et donne exactement les notifications et les emails prévus par la matrice. Y compris après un redémarrage du courtier.
5. **Le moniteur externe armé et vérifié**. Les quatre battements sortants du runbook sont déclarés, au moins un signal a été reçu par le moniteur pour chacun, et une coupure volontaire d'une tâche a bien déclenché son alerte.

Deux conditions de blocage automatique, quelle que soit la suite des verdicts :

- un événement d'outbox **parqué** en dehors des scénarios de recette qui le provoquent volontairement (CRON-RELAIS-5, CRON-PURGEOUT-2) — cela signifie qu'un événement réel a été perdu ;
- une tâche dont le battement est absent alors que son horaire est passé, sans coupure explicite et documentée — c'est la panne que ce cahier existe pour attraper.

9.4 Nettoyage après campagne

```
# Retirer les lignes d'outbox de recette
npx tsx --env-file=.env -e 'import p from "./packages/libs/prisma";
Promise.all([p.outboxEvent.deleteMany({where:{correlationId:"seed-
outbox"}}),p.outboxEvent.deleteMany({where:{correlationId:"recette-parque"}}])).then(r=>
{console.log(r);process.exit(0)})'
```

```
# Remettre les paramètres aux valeurs par défaut
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-settings.ts
```

```
# Remettre le jeu d'essai à zéro
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-deals.ts
```

```
# Effacer les verrous d'alerte
npx tsx --env-file=.env scripts/recette-alertes-verrou.ts --reset
```

```
# Vider la boîte de recette
curl -s -X DELETE http://localhost:8025/api/v1/messages
```

Et, dans `.env`, retirer toutes les variables `*_CRON_ENABLED=false`, `OUTBOX_RELAY_ENABLED=false`, `MESSAGING_RELAY_ENABLED=false`, `NOTIFICATION_CONSUMER_ENABLED=false` et la carte `CRON_HEARTBEAT_PING_URLS` de test posées pendant la campagne. Une variable de coupure oubliée est exactement la panne silencieuse que ce cahier cherche à éviter.

Les scripts `scripts/recette-*.ts` créés pour la campagne sont des outils de recette : soit ils sont versionnés délibérément, soit ils sont supprimés à la fin. Ils ne doivent pas rester non suivis dans l'arbre de travail.

Fin du cahier de recette — tâches planifiées, relais d'événements et consommateurs.